

Module II

History of Deep Learning, Deep Learning Success Stories, McCulloch Pitts Neuron, Thresholding Logic, Perceptrons, Perceptron Learning Algorithm, Deep feedforward Networks, Multilayer Perceptrons (MLPs), Representation Power of MLPs, Sigmoid Neurons, Gradient Descent, Representation Power of Feedforward Neural Networks, Backpropagation

Activation/ Transfer Functions

$$Sum_k = \sum_{j=0}^n W_{kj} I_j + b_k$$

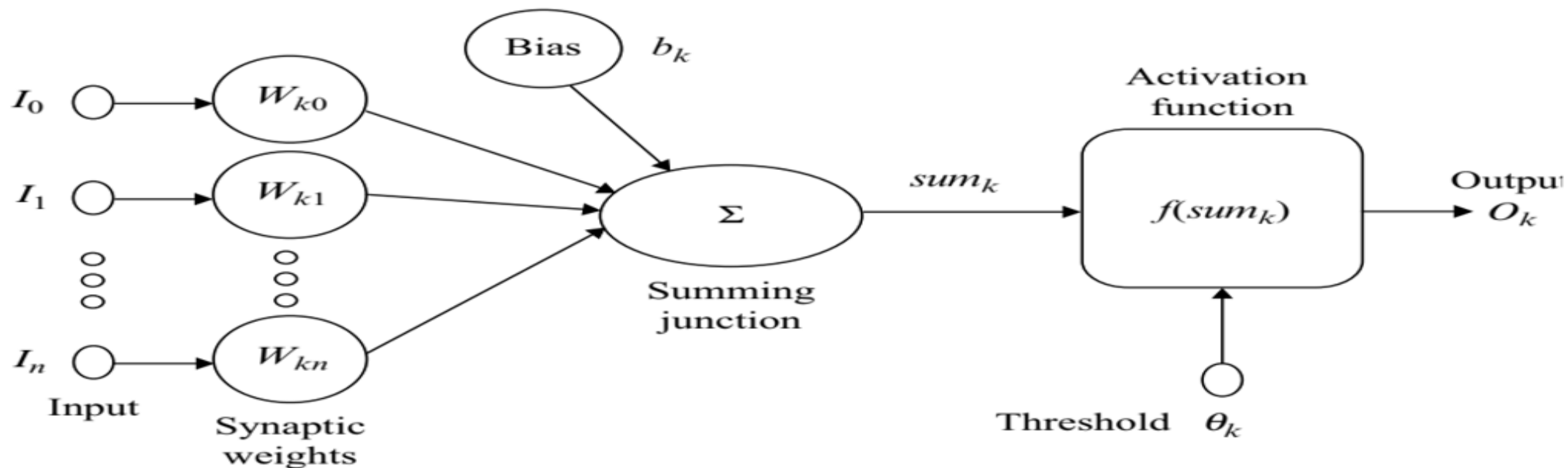


Fig. 2.10 A simple ANN

Example

- weight vector = $w = [0.2; 0.3; 0.9]$
- Bias = $b = 0.5$
- Input vector = $x = [0.5; 0.6; 0.1]$

$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}} = \frac{1}{1 + e^{-(.5*.2 + .6*.3 + .1*.9 + .5)}} = \frac{1}{1 + e^{-0.87}} = .70$$

Activation/ Transfer Functions

- The **sigmoid activation** function, whose graph is **s-shaped**,
- used in the construction of ANNs.
- It is defined as a strictly increasing function that exhibits a graceful balance between linear and non-linear behaviour.
- It passes negative information when the output is less than the threshold θ_k
- positive information when the output is greater than the threshold θ_k .
- **the activation function is a continuous function that varies gradually between two asymptotic values, typically 0 and 1, or -1 and +1.**
- **The commonly used activation function is the logistic function,**

Activation/ Transfer Functions

- $f(\text{Sum}_k) = \frac{1}{1 + e^{-sX}(\text{sum}_k)}$
- where 's' is a slope parameter of the sigmoid function that adjusts the abruptness of this function as it changes between the two asymptotic values.
- By varying the parameters, we can obtain sigmoid functions of different slopes.

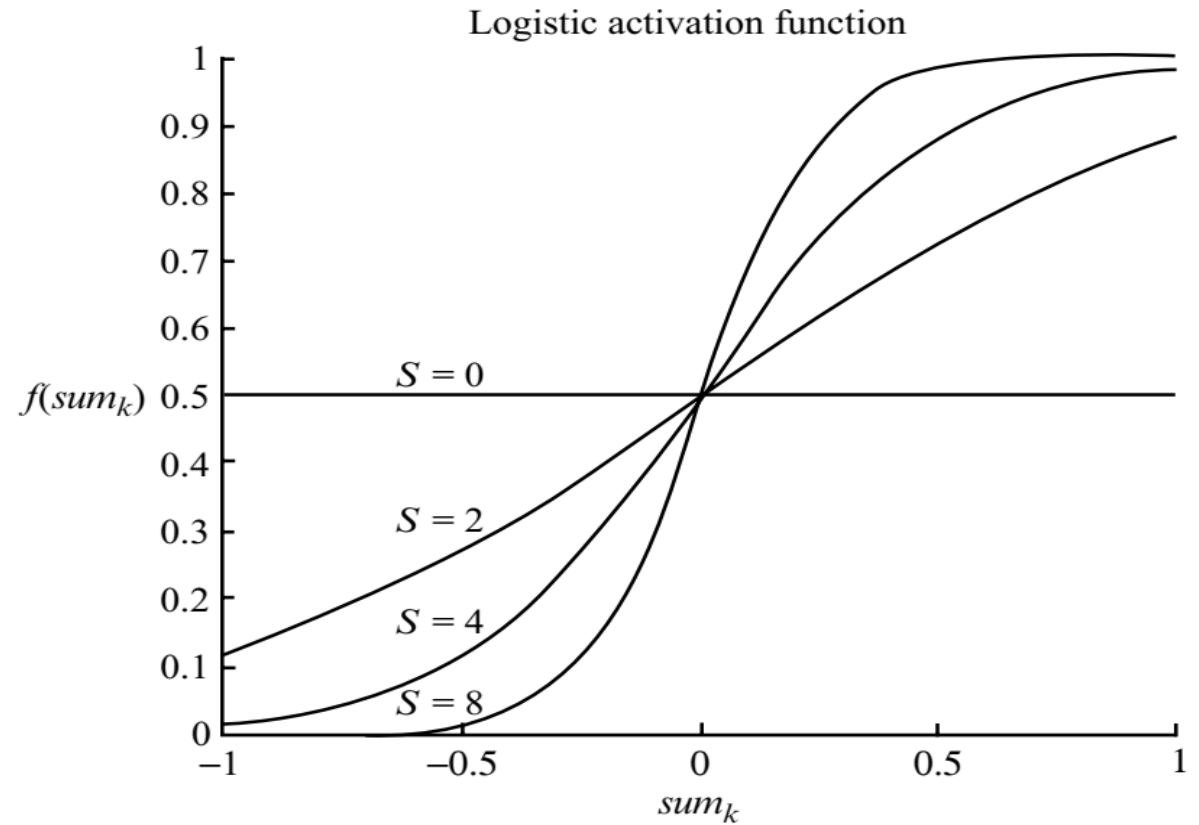


Fig. 2.17 Logistic activation function

Activation/ Transfer Functions

Function	Equation
Identity function (purelin)	$f(\text{sum}_k) = \text{sum}_k \text{ for all } \text{sum}_k$
Binary step function	$f(\text{sum}_k) = \begin{cases} 1 & \text{if } \text{sum}_k \geq \theta_k \\ 0 & \text{if } \text{sum}_k < \theta_k \end{cases}$
Binary sigmoid (logsig)	$f(\text{sum}_k) = \frac{1}{1 + \exp(-s \times \text{sum}_k)}$
Bipolar sigmoid function (tansig)	$f(\text{sum}_k) = \frac{1 - \exp(-s \times \text{sum}_k)}{1 + \exp(-s \times \text{sum}_k)}$
Hard limit (hardlim)	$f(\text{sum}_k) = \begin{cases} 1 & \text{if } \text{sum}_k \geq 0 \\ 0 & \text{if } \text{sum}_k < 0 \end{cases}$
Symmetrical hard limit (hardlims)	$f(\text{sum}_k) = \begin{cases} 1 & \text{if } \text{sum}_k > 1 \\ \text{sum}_k & \text{if } 0 \leq \text{sum}_k \leq 1 \\ 0 & \text{if } \text{sum}_k < 0 \end{cases}$

Activation/ Transfer Functions

Function	Equation
Saturating linear (satlin)	$f(sum_k) = \begin{cases} 1 & \text{if } sum_k \geq 0 \\ -1 & \text{if } sum_k < 0 \end{cases}$
Symmetric saturating linear (satlins)	$f(sum_k) = \begin{cases} 1 & \text{if } sum_k > 1 \\ sum_k & \text{if } -1 \leq sum_k \leq 1 \\ -1 & \text{if } sum_k < -1 \end{cases}$

Activation/ Transfer Functions

- Radial basis functions are special class of activation functions used in RBF NNs, which are extremely efficient as universal function approximators.

Table 2.4 Radial basis/specialized activation functions

Function	Equation
Gaussian	$f(\text{sum}_k) = \exp\left(-\frac{\ \text{sum}_k - c_k\ ^2}{2\sigma^2}\right)$
Multiquadratics	$f(\text{sum}_k) = \sqrt{\ \text{sum}_k - c_k\ ^2 + a^2}$
Inverse multiquadratics	$f(\text{sum}_k) = \left(\ \text{sum}_k - c_k\ ^2 + a^2\right)^{-1/2}$

Note: Where c_k is the vector representing the function centre and a and σ are parameters affecting the spread of the radius

Example

- A single input, single output neuron has a weight of 5 and a bias of -3. What is the net input to the transfer function if it is given an input of 1? What will be the output if the transfer function is bipolar sigmoid given slope $s = 0.5$?

$$sum_k = 5 \times 1 - 3 = 2$$

The output is given by

$$f(sum_k) = \frac{1}{(1 + \exp(-s \times sum_k))}$$

$$f(sum_k) = 1/(1 + \exp(-0.5 \times 2)) = 0.731$$

Example

- Q2 A 2-input single-output NN has weight values $[1.3 \ 2.7]$ and bias of 1.6. It is given an input $[3.1 \ 2.4]^T$. What is the output if the identity function is used as the transfer function?
- $\text{sumk} = 1.3 \times 3.1 + 2.7 \times 2.4 + 1.6 = 12.11$
- The output is given by $f(\text{sumk}) = \text{sumk} = 12.11$
- For the network in Question 2 , consider a symmetric saturating linear function to be used as activation. What will be the output if the input is $[-6 \ 4.1]^T$?
- Solution: $\text{sumk} = 1.3 \times (-6) + 2.7 \times 4.1 + 1.6 = 4.87$ The output is given by $f(\text{sumk}) = 1$ as $\text{sumk} > 1$

Example

- A 3-input 2-output NN has the weight values $w_{11} = 0.6$, $w_{12} = 1.1$, $w_{21} = 0.7$, $w_{22} = 0.5$, $w_{31} = 0.8$, and $w_{32} = 0.2$. It is given an input of $[0.3 \ 0.7 \ 1.6]$. What is the output of the NN if the binary step function is used? Assume a threshold of 1.5.
- Solution: Since there are 2 output units, $k = 1, 2$
- $\text{sum}_{k=1} = w_{11} \times I_1 + w_{21} \times I_2 + w_{31} \times I_3 = 0.6 \times 0.3 + 0.7 \times 0.7 + 0.8 \times 1.6 = 1.95$
- $\text{sum}_{k=2} = w_{12} \times I_1 + w_{22} \times I_2 + w_{32} \times I_3 = 1.1 \times 0.3 + 0.5 \times 0.7 + 0.2 \times 1.6 = 1$
- The output is given by $f(\text{sum}_{k=1}) = 1$ as $\text{sum}_{k=1} > 1.5$ (threshold)
 $f(\text{sum}_{k=2}) = 0$ as $\text{sum}_{k=2} < 1.5$ (threshold)

Example

Find out the undecided band for the perceptron OR from the results obtained .

At the end of training, the weights and bias are $w_1 = 2$, $w_2 = 2$ and $b = -1$ The threshold for the network is set at 0.5

Solution:

The positive region is given by $2 \times I_1 + 2 \times I_2 - 1 > 0.5$

The negative region is given by $2 \times I_1 + 2 \times I_2 - 1 < -0.5$

The region between these two planes is the **undecided band**

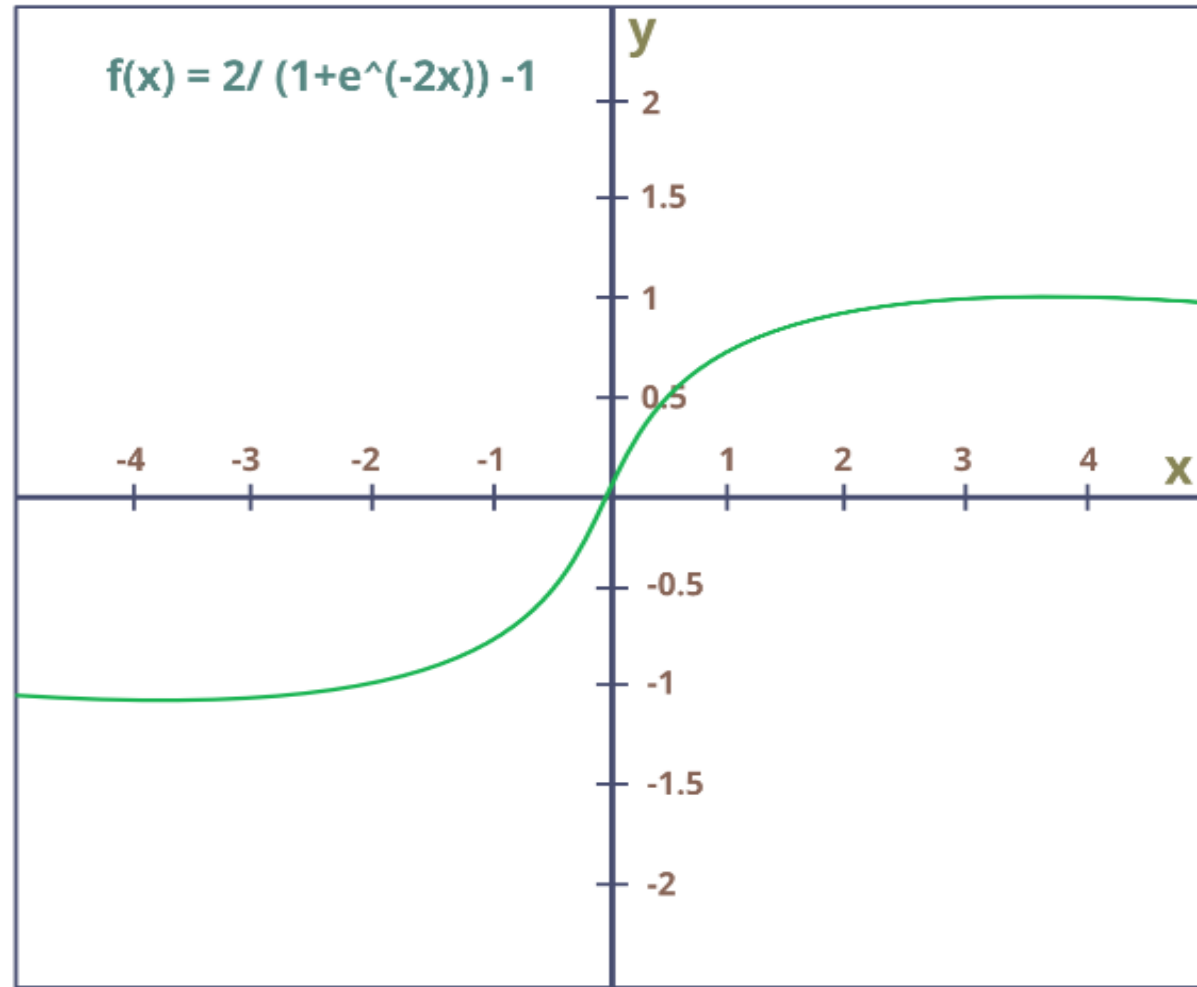
i.e., $2 \times I_1 + 2 \times I_2 - 1 < 0.5$,

$2 \times I_1 + 2 \times I_2 - 1 > -0.5$

or $2 \times I_1 + 2 \times I_2 < 1.5$,

$2 \times I_1 + 2 \times I_2 > 0.5$

Activation function- Tangent Hyperbolic function



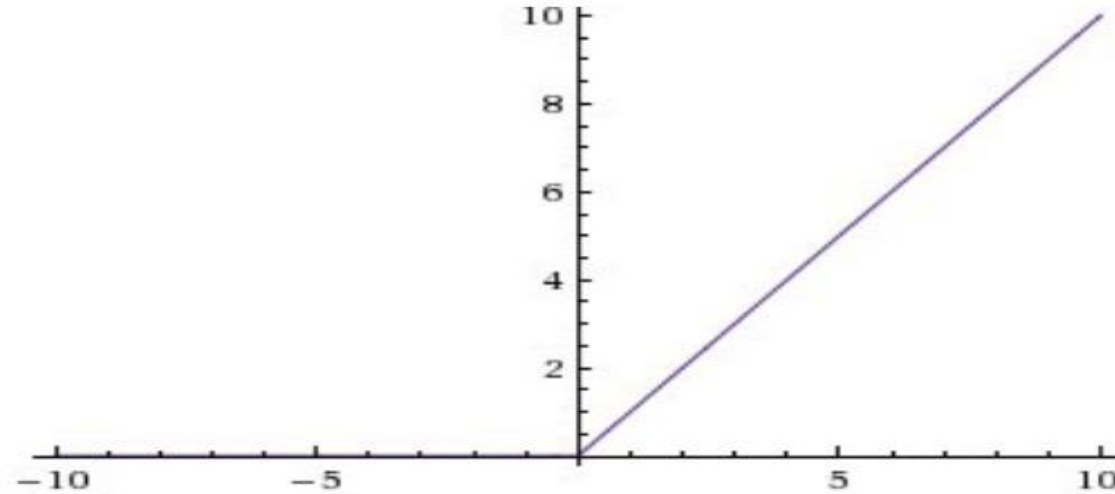
Activation function

- better than sigmoid function is Tanh function (**Tangent Hyperbolic fun**).
- It's **shifted version of the sigmoid function**.
- Both are similar and can be derived from each other.
- **Value Range** :- -1 to +1
- **Nature** :- non-linear
- **Uses** :- Usually used in **hidden layers of a neural network** as it's values lies between **-1 to 1**
- hence the mean for the hidden layer comes out be 0 or very close to it, hence helps in *centering the data* by bringing mean close to 0.

$$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1$$

Activation function –RELU (*Rectified linear unit*)

RELU Function



- most widely used activation fun
- implemented in ***hidden layers*** of Neural network.
- **Equation :- $A(x) = \max(0, x)$.**
- **It gives an output x if x is positive and 0 otherwise.**

Activation function –RELU (*Rectified linear unit*)

- **Value Range** :- $[0, \infty)$
- **Nature** :- non-linear, which means we can easily backpropagate the errors and have multiple layers of neurons being activated by the ReLU function.
- **Uses** :- **ReLU is less computationally expensive than tanh and sigmoid** because it involves simpler mathematical operations.
- At a time only a few neurons are activated making the network **sparse** making it efficient and **easy for computation**.
- In simple words, RELU learns *much faster* than sigmoid and Tanh function.

Activation function –Elu function:

The **exponential Linear Unit (ELU)** is also similar to **Leaky Relu** but differs for negative input.

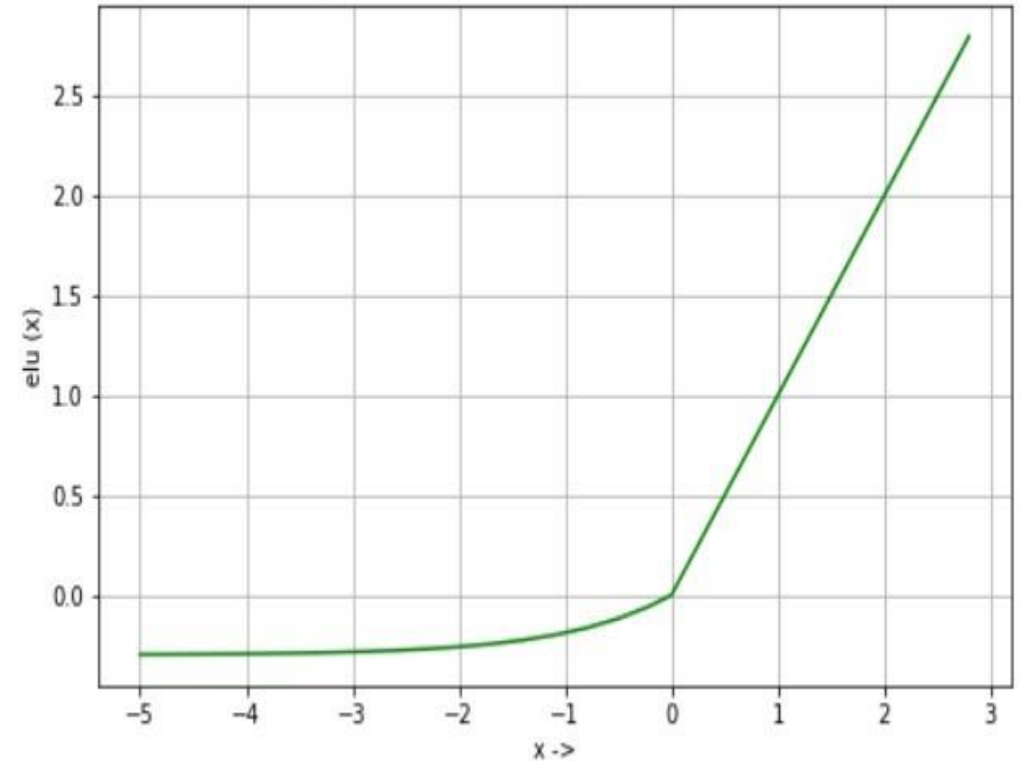
It also helps to **overcome the dying neuron problem**.

$\text{elu}(x) = x$ if $x > 0$

$\text{elu}(x) = \alpha * (\exp(x) - 1)$ if $x < 0$

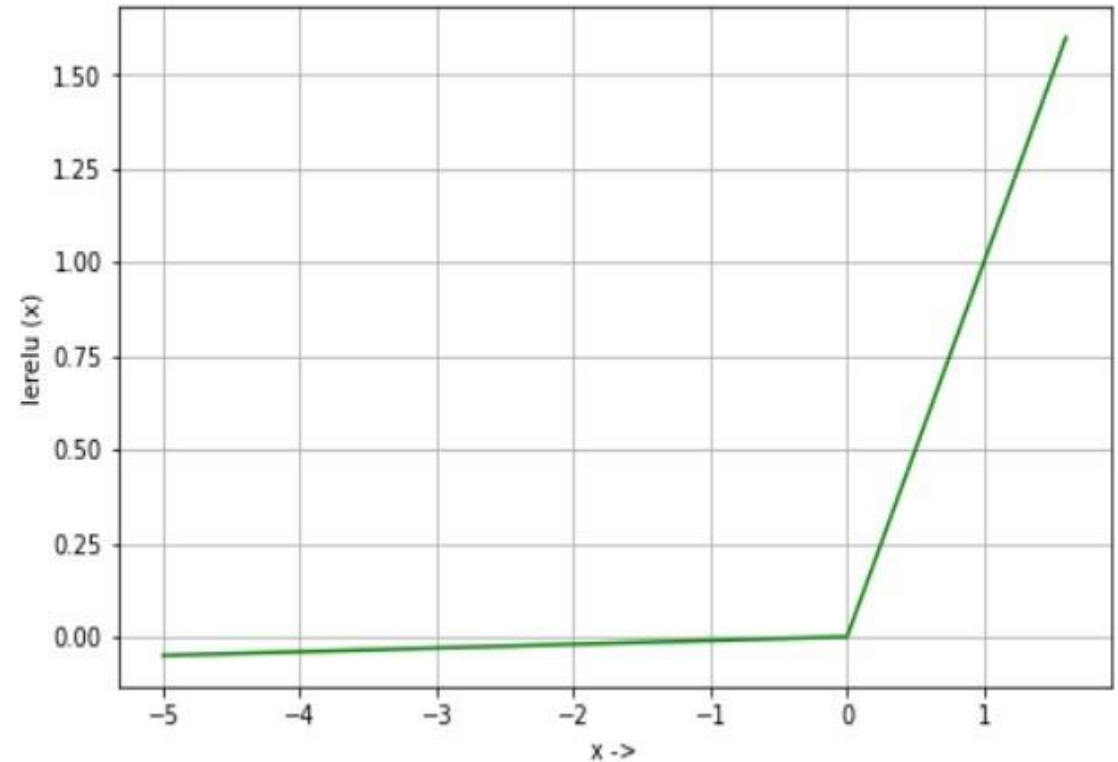
Uses: It has the same purpose that of Leaky Relu and convergence of cost function towards zero is faster than Relu as well as Leaky Relu.

For example, **neural network learning on Imagenet using Elu is faster than using Relu**.



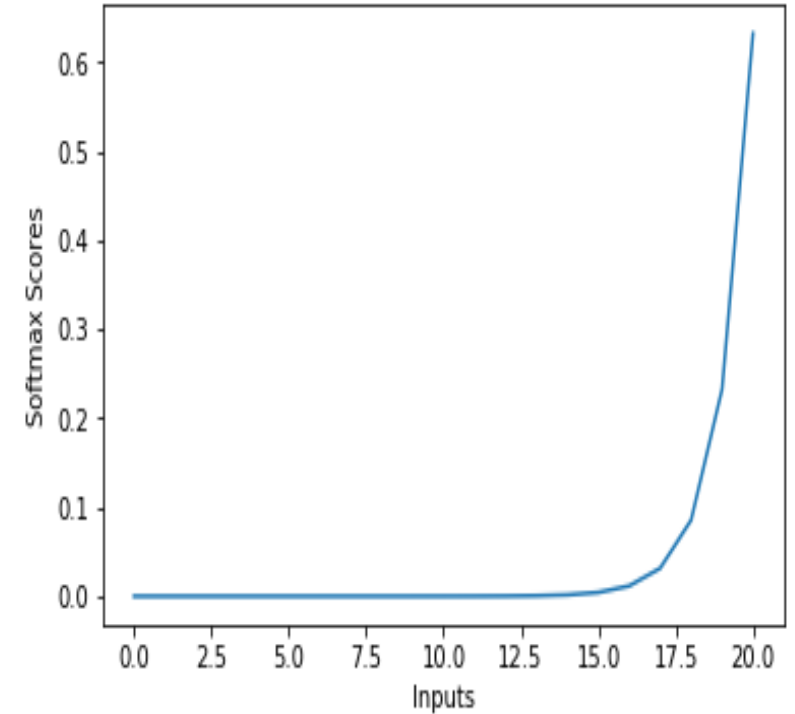
Activation function – Leaky RELU

- **Leaky Rectified linear unit(Leaky Relu)** is an extension of the Relu function to overcome the dying neuron problem.
- $\text{lerelu}(x) = x$ if $x > 0$
- $\text{lerelu}(x) = 0.01 * x$ if $x \leq 0$
- **Uses:** Relu return 0 if the input is negative and hence the neuron becomes inactive as it does not contribute to gradient flow.
- Leaky Relu overcomes this problem by allowing small value to flow when the input is negative.
- So, if the learning is too slow using Relu, one can try using Leaky Relu to see any improvement happens or not.



Activation function –Soft Max

- The softmax function is also a type of sigmoid function but is handy when we are trying to handle **multi- class classification problems**.
- **Nature :-** non-linear
- **Uses :-** Usually used when trying to handle multiple classes., commonly found in the **output layer of image classification problems**.
- The softmax function would squeeze the outputs for each class **between 0 and 1** and would also divide by the sum of the outputs.
- **Output:-** The softmax function is ideally used in the output layer of the classifier where we are actually trying to attain the **probabilities to define the class of each input**.



Activation function

- The basic rule of thumb is if you really don't know what activation function to use, then simply use ***RELU*** as it is a general activation function in hidden layers and is used in most cases these days.
- If your output is for **binary classification** then, ***sigmoid function*** is very natural choice for **output layer**.
- If your output is for **multi-class classification** then, **Softmax** is very useful to predict the probabilities of each classes.

First Generation Neural Networks

- **McCulloch-Pitts threshold neurons**
- a neuron sends a **binary ‘high’ signal** if the sum of its weighted incoming signals rises **above a threshold value**.
- Even though these neurons can only give digital output, they have been successfully applied in **powerful ANNs such as multi-layer perceptrons and Hopfield nets**.
- For example, any function with Boolean output can be computed by a multi-layer perceptron with a single hidden layer;

McCulloch-Pitts Neuron Model

- The following are the assumptions involved in this model:
- (a) McCulloch neurons have a binary output of **either 'one' or 'zero'**.
- If the **firing of neuron takes place**, then the neuron has an activation of **'one'** and if it **does not fire**, then the neuron has an activation of **'zero'**.
- (b) McCulloch neurons are connected via the connection links called the 'weight path'.
- Each of the weight paths has some values and is often considered as the strength of the NNs.
- (c) If the weight value on the path is **positive**, then the connection path is **excitatory**;
- or **else, it is inhibitory**.
- Same weight values are kept for the excitatory connections

McCulloch-Pitts Neuron Model

- (d) A threshold level is present in each of the neuron.
- If the **net input passing through the neuron** is greater than the **threshold** level, then the neuron fires an output .
- However, the threshold value θ_k should satisfy the constraint given in Eq. such that the inhibition is absolute.
- (e) The time taken for a signal to pass through a connection link is a one-time step

McCulloch-Pitts Neuron Model

- The excitatory connections has $w > 0$ and the inhibitory connection has weight values as $-v$ and $v > 0$.
- Here, **n units have excitatory connections**
- and **m connections have inhibitory connections.**
- The output 'O' of the McCulloch-Pitts neuron is given by the activation function as in Eq.
- Here, Sum_k is the total weighted net input and θ_k is the threshold level of the neuron.

$$f(sum_k) = \begin{cases} 1 & \text{if } sum_k \geq \theta_k \\ 0 & \text{if } sum_k < \theta_k \end{cases}$$

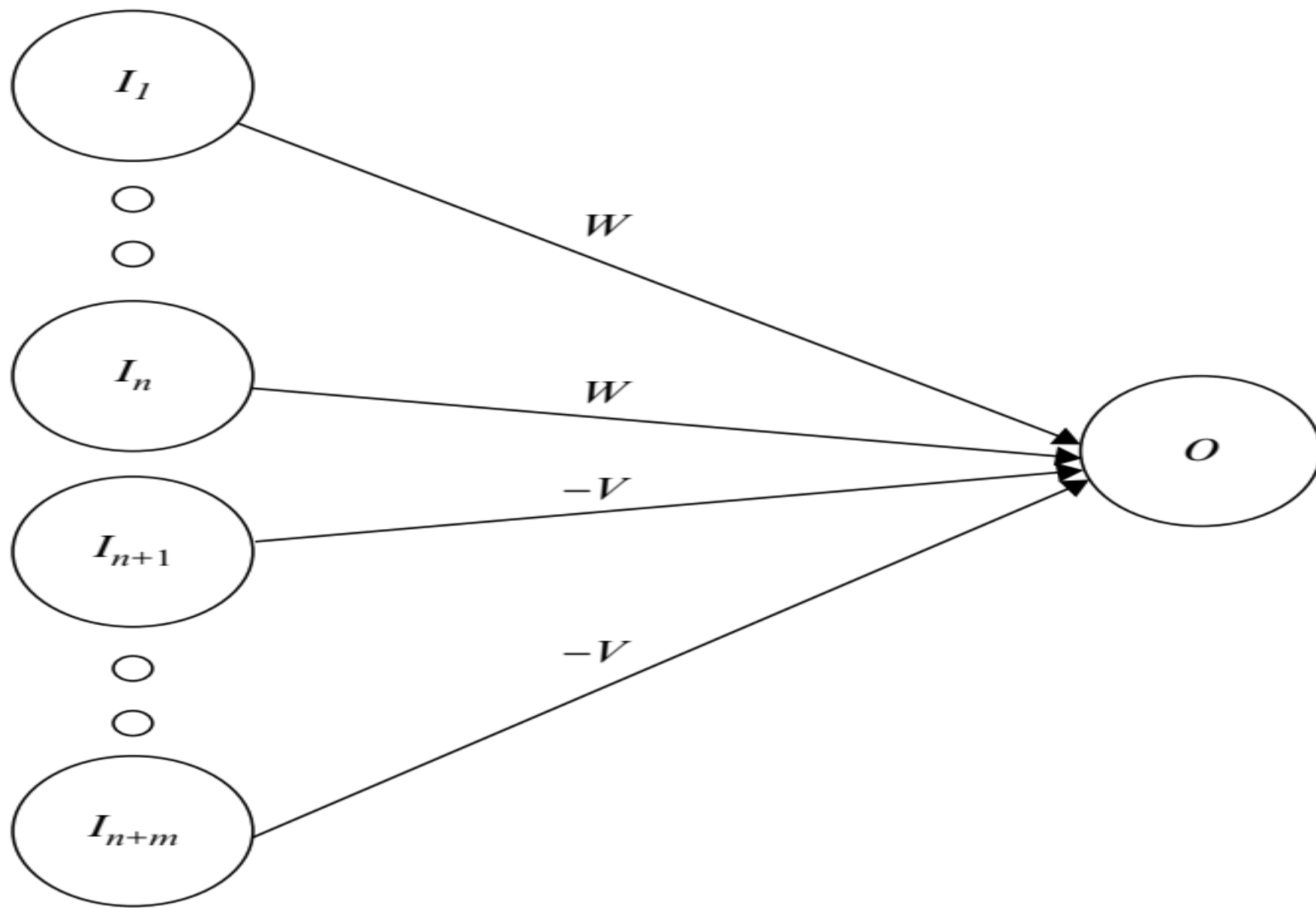


Fig. 2.18 Architecture of a simple McCulloch-Pitts neuron

The threshold value

$$\theta_k > nXw - p$$

Let us try to solve some simple Boolean AND and OR logic functions.

Let us consider two binary inputs

and value of the threshold θ_k be fixed as 2.

In addition, the weight values connecting the two inputs to the output neuron be **'1' for AND** logic and **'2' for OR logic**.

The following are the truth table of AND and OR logic function performed on a **2-input, 1-output McCulloch-Pitts NN**

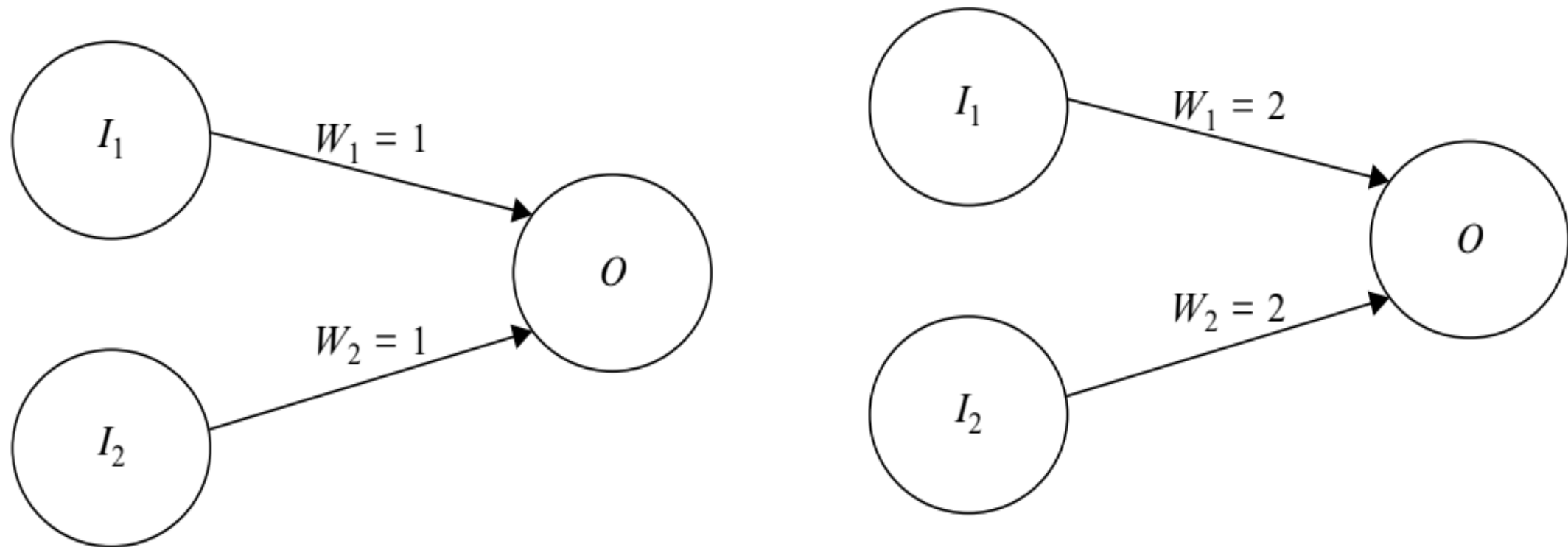


Fig. 2.19 AND-OR logics performed by McCulloch-Pitts neuron

Table 2.5 AND-OR logics performed using McCulloch-Pitts NN

Input-1(I_1)	Input-2(I_2)	AND logic weights ($W_1=1, W_2=1$) sum_k	OR logic weights ($W_1=2, W_2=2$) sum_k	Compare sum_k and $\theta_k = 2$ using Eq. (2.5)	AND logic Output (O)	OR logic Output (O)
0	0	0	0		0	0
0	1	1	2		0	1
1	0	1	2		0	1
1	1	2	4		1	1

Learning Rules : Hebbian and Delta

- thinking in the right direction is a learning strategy or a learning rule.
- The Hebb rule given
- $w_{jk}(t) = w_{jk}(t - 1) + n \times O_k(t) \times I_j(t)$
- Where, j and k are the indices of input and output units, and t is the iteration number.

Learning Rules : Hebbian and Delta

- Let us represent '1' for a condition to happen
- and '0' for a condition that does not happen.
- The condition can be a stimulus or a response.
- Here **$W0 = 1$** and **bias (b) = -0.5** is fixed and is not allowed to change during the training phase.
- **The bias is set such that the network will respond (i.e., $O = 1$) even if the unconditional stimulus (presentation of food) is present or absent.**

Learning Rules : Hebbian and Delta

- the value of W_0 should be greater than the bias value.
- Let the value of the learning factor, $n = 1$.
- **If the inputs are $I_0 = 1$ and $I_1 = 1$, then both the presentation of food and the ringing of the bell occurs together.**
- **If $I_0 = 0$ and $I_1 = 1$, then the food is not presented and the bell rings.**

Learning Rules : Hebbian and Delta

t=0	I0	I1	$W_o = 1$	W=1	Sum Calculation $(W_o \times I_o(t) + W(t-1) \times I_1(t) - b)$	$\Theta=0$ hard limit	O	$W_{jk}(t)$ $= W_{jk}(t-1) + \eta \times O_k(t) \times I_j(t)$
1	0	1	$W_o = 1$	$W(0) = 0$	$(1 \times 0 + 0 \times 1) - 0.5 = -0.5$	$-0.5 < \Theta$	$O(1) = 0$	$W(1) = 0 + 1 \times 0 \times 1 = 0$
2	1	1	$W_o = 1$	$W(1) = 0$	$(1 \times 1 + 0 \times 1) - 0.5 = 0.5$	$0.5 > \Theta$	$O(2) = 1$	$W(2) = 0 + 1 \times 1 \times 1 = 1$
3	0	1	$W_o = 1$	$W(2) = 1$	$(1 \times 0 + 1 \times 1) - 0.5 = 0.5$	$0.5 < \Theta$	$O(3) = 1$	$W(3) = 1 + 1 \times 1 \times 1 = 2$

Learning Rules : Hebbian and Delta

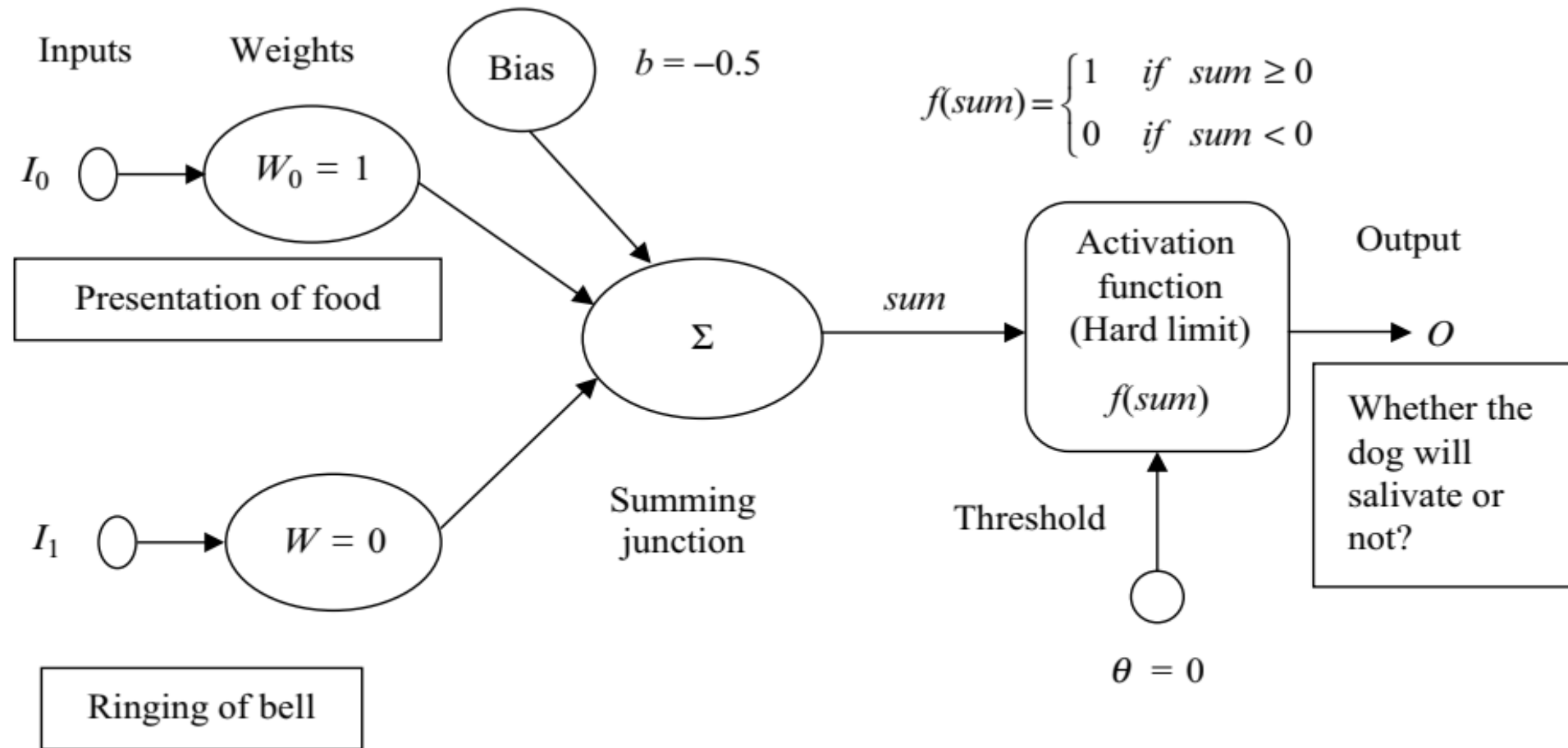


Fig. 2.20 A simple Hardlimit NN

Learning Rules : Hebbian and Delta

During the first iteration ($t = 1$), one of the input stimuli (presentation of food, $I_0 = 0$) is not present and the response seen is that the dog did not salivate, $O = 0$.

In the second iteration, when both the input stimulus are present (presentation of food, $I_0 = 1$; ringing the bell, $I_1 = 1$), it is observed that the dog has salivated, $O = 1$.

Now it should be noted from the third iteration that, even in the absence of the input stimuli (presentation of food, $I_0 = 0$), the dog is made to salivate by obtaining the desired response, $O = 1$.

Variation of Hebbian Learning

- The previous Hebbian rule which is given in Eq. (2.7) has certain limitations.
- When we continue presenting the input patterns for more number of iterations, there is a problem with regard to the **weight values becoming arbitrarily large**.
- In addition, there is no mechanism to control the decrease of weights.
- Therefore to overcome the limitation, **Hebb rule** with a decay factor γ is proposed
- $$\begin{aligned}w_{jk}(t) &= w_{jk}(t-1) + n * O_k(t) * I_j(t) - \gamma * w_{jk}(t-1) \\ &= (1 - \gamma)w_{jk}(t-1) + n X O_k(t) X I_j(t)\end{aligned}$$

Here, γ is a constant which is positive and lesser than unity.

Variation of Hebbian Learning

When γ becomes zero, the learning law becomes the standard one which is same as in Eq.

However, when γ becomes unity, the rule quickly forgets the past inputs and memorizes the present input patterns.

Suppose the change in weights is replaced by the desired output with the difference between the desired and the actual output, then a new kind of learning rule called delta learning rule emerges.

Delta Learning Rule

- The initial delta rule when developed for a single output unit is computed such that it will minimize the difference between the weighted sum that is given into the output unit and target value or desired output.
- Here, the error is minimized over all the training patterns.
- The error reduction is carried out by each input vector, one at a time.
- Let us consider a single neuron with n number of units.
- Let I be the input vector with **n number of elements** and D is the desired output.
- Let j be the index of the input elements and W is the weights from input units to the output unit O .

Delta Learning Rule

- $Sum = \sum_{j=1}^n I_j W_j$
- The squared error (SE) for an input vector I is given as
- $E = (D - Sum)^2$
- where E is a function of all of the weights and $W_j, j = 1, 2, 3, \dots, n$
- The direction of the error (E) is given by the gradient of E of the vector consisting of the partial derivatives of E with respect to each of the weights.
- The gradient $\frac{\delta E}{\delta W_j}$, $j=1..n$ gives the direction of the increase in error
- The gradient $-\frac{\delta E}{\delta W_j}$, $j=1..n$ gives the direction of the decrease in error

Delta Learning Rule

We know $sum = \sum_{j=1}^n I_j W_j$, therefore

$$\begin{aligned}\frac{\partial E}{\partial W_{j=1..n}} &= -2 \times (D - sum) \times \frac{\partial(sum)}{\partial W_{j=1..n}} \\ &= -2 \times (D - sum) \times I_{j=1..n}\end{aligned}$$

Where $E = (D - Sum)^2$

Therefore, for a given learning rate of η , the error E will be reduced by modifying the weights according to the delta rule given in Eq.

$$\Delta W_{j=1..n} = \eta \times (D - sum) \times I_{j=1..n}$$

Delta Learning Rule

- The delta rule for more than one unit is considered by having m number of output units and let k be the index of the output elements.
- $E = \sum_{k=1}^m (D_j - Sum_k)^2$
- Similar to the derivation of the delta rule for a single-layer output unit, the error E can be reduced by adjusting the weights
- $W_{jk}, j = 1..n, k = 1..m$ in the opposite direction of the gradient

$$- \frac{\delta E}{\delta W_{jk}}, j=1..n, k=1..m$$

Delta Learning Rule

Now the gradient to be adjusted with weights can be derived as follows:

$$\begin{aligned}\frac{\partial E}{\partial W_{jk}} &= \frac{\partial E}{\partial W_{jk}} \sum_{k=1}^m (D_k - sum_k)^2 \\ &= \frac{\partial E}{\partial W_{jk}} \times (D_k - sum_k)^2\end{aligned}$$

Delta Learning Rule

It should be noted that the weights W_{jk} influence the error at the output units O_k and is given by

$$O_k = \sum_{j=1}^n I_j W_{jk}$$

Therefore,

$$\begin{aligned} \frac{\partial E}{\partial W_{jk}} &= -2 \times (D_k - \text{sum}_k) \times \frac{\partial(\text{sum}_k)}{\partial W_{jk}} \\ &= -2 \times (D_k - \text{sum}_k) \times I_j \end{aligned}$$

Delta Learning Rule

Therefore, for a given learning rate of η , the error E will be reduced by modifying the weights according to the delta rule given as:

$$\Delta W_{jk} = \eta \times (D_k - \text{sum}_k) \times I_k$$

$$E = \sum_{k=1}^m (D_j - O_k)^2$$

Now the gradient to be adjusted with weights can be derived as:

$$\begin{aligned} \partial E / \partial W_{jk} &= \partial E / \partial W_{jk} \sum_{k=1}^m (D_k - O_k)^2 \\ &= \partial E / \partial W_{jk} \times (D_k - O_k)^2 \end{aligned}$$

Delta Learning Rule

$$\begin{aligned}\frac{\partial E}{\partial W_{jk}} &= -2 \times (D_k - O_k) \times \frac{\partial(\text{sum}_k)}{\partial W_{jk}} \\ &= -2 \times (D_k - O_k) \times I_j \times f'(\text{sum}_k)\end{aligned}$$

Now the error E is reduced by modifying the weights according to the extended delta rule

$$\Delta W_{jk} = \eta \times (D_k - O_k) \times I_j \times f'(\text{sum}_k)$$

.

Gradient Descent Learning

- Designing and training a neural network is not much different from training any other machine learning model with gradient descent.
- how to build a machine learning algorithm by specifying an optimization procedure, a cost function, and a model family.
- **The largest difference between the linear models and neural networks is that the nonlinearity of a neural network causes most interesting loss functions to become non-convex.**
- NN are usually trained by using **iterative, gradient-based optimizers**
- drive the cost function to a very low value,
- **linear equation solvers used to train linear regression models or the convex optimization algorithms with global convergence guarantees** used to train logistic regression or SVMs.

Gradient Descent Learning

- Convex optimization converges starting from any initial parameters.
- **Stochastic gradient descent applied to non-convex loss functions** has no such convergence guarantee,
- and is sensitive to the values of the initial parameters.
- For feedforward neural networks, it is important to initialize all weights to small random values.
- The biases may be initialized to zero or to small positive values.
- The **iterative gradient-based optimization** algorithms used to train feedforward networks

Gradient Descent Learning

- We can train models such as linear regression and support vector machines with gradient descent too,
- and in fact this is common when the training set is extremely large.
- Computing the gradient is slightly more complicated for a neural network

Gradient Descent Learning

- **Cost Functions**
- our parametric model defines a distribution $p(\mathbf{y}|\mathbf{x}; \boldsymbol{\theta})$ and
- we simply use the **principle of maximum likelihood**.
- This means we use the **cross-entropy between the training data and the model's predictions** as the cost function.
- The **weight decay approach** used for linear models is also directly applicable to deep neural network

Gradient Descent Learning

- **Cost Functions**
- **Learning Conditional Distributions with Maximum Likelihood.**
- cost function is simply the **negative log-likelihood** as the cross-entropy between the training data and the model distribution.
- $J(Q) = -E_{x \sim \hat{p}_{data}} \log p_{model}(y|x)$
- A gradient is nothing but a derivative that defines the effects on outputs of the function with a little bit of variation in inputs.

Gradient Descent Learning

- Gradient Descent (GD) is a widely used optimization algorithm in deep learning
- that is **used to minimize the cost function** of a neural network model during training.
- It works by iteratively adjusting the weights or parameters of the model in the direction of the **negative gradient** of the cost function
- until the minimum of the cost function is reached.

- gradient descent aims at minimizing some objective function:
- $\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$
- Thus to find a local minimum of a function using gradient descent, one takes steps proportional to the negative of the gradient (or of the approximate gradient) of the function at the current point.
- In contrast, if one takes steps proportional to the positive of the gradient, one approaches a local maximum of that function; the procedure is then known as gradient ascent.
- gradient ascent aims at maximizing some objective
- $\theta_j \leftarrow \theta_j + \alpha \frac{\partial}{\partial \theta_j} J(\theta)$

- If I have a function which is convex then at the bottom the gradient or derivative is 0.
- Similarly, if we have a concave function at the top gradient or derivative is 0.

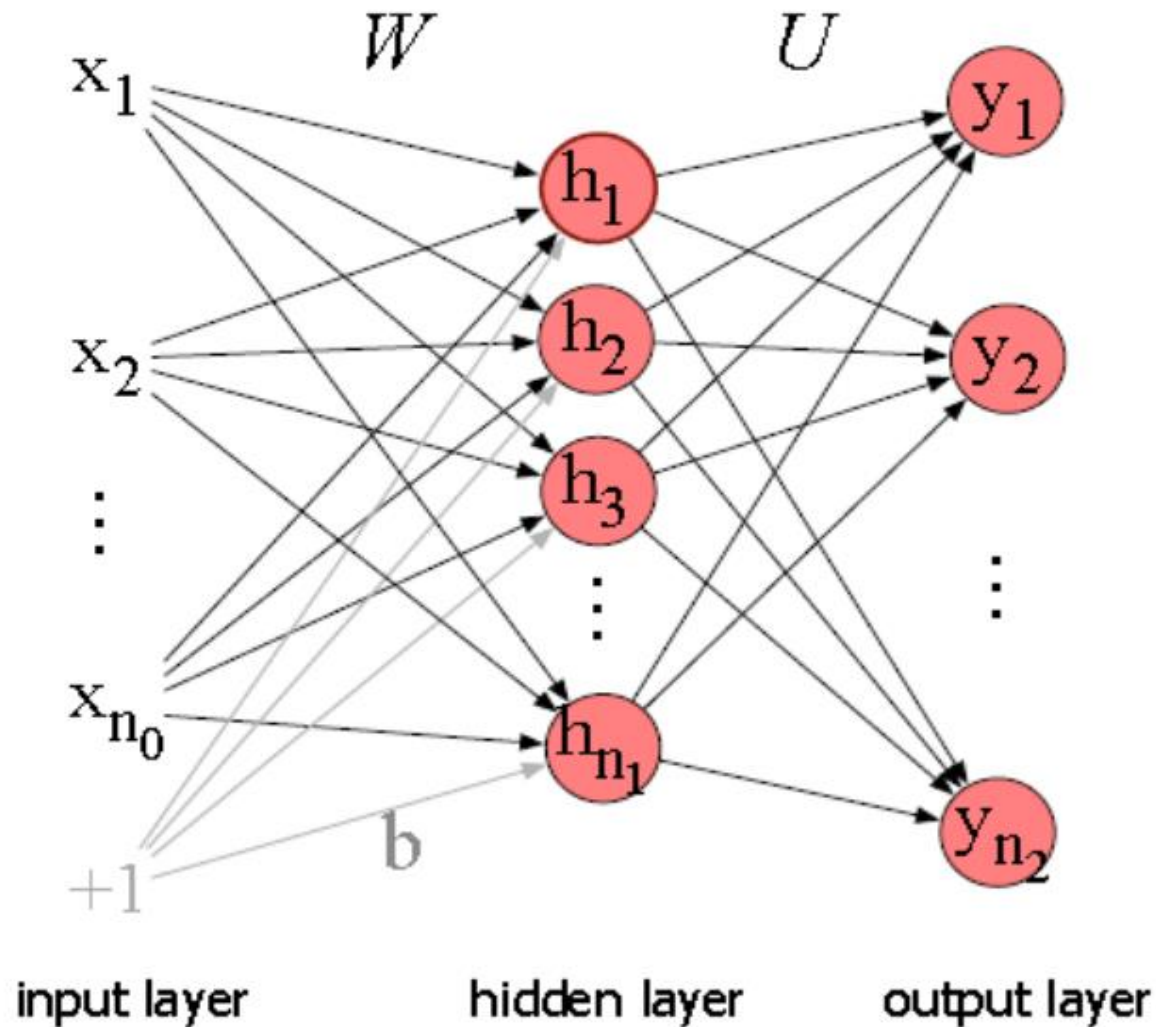
Why are we interested in 0?

- This is because it helps us find either the lowest(convex) or highest(concave) value of the function
- Our machine learning algorithm, e.g. logistic regression, has a cost function and they can either be concave or convex.
- If it is convex we use Gradient Descent
- and if it is concave we use we use Gradient Ascent.

Feedforward Neural Networks

- A feedforward network is a **multilayer network**
- network in which the units are connected with no cycles;
- outputs from units in each layer are passed to units in the next higher layer,
- **and no outputs are passed back to lower layers.**
- feedforward networks have three kinds of nodes:
input units, hidden units, and output units.
- perceptrons are purely linear, but modern networks are made up of units with non-linearities like sigmoids

Feedforward Neural Networks



A simple 2-layer feedforward network, with one hidden layer, one output layer, and one input layer

Feedforward Neural Networks

- The input layer x is a vector of simple scalar values.
- hidden layer h formed of hidden units h_i ,
- taking a weighted sum of its inputs and **then applying a non-linearity**.
- fully-connected, meaning that each unit in each layer takes as input the outputs from all the units in the previous layer,
- and there is a link between every pair of units from two adjacent layers.
- Thus each hidden unit sums over all the input units

Feedforward Neural Networks

computation only has three steps:

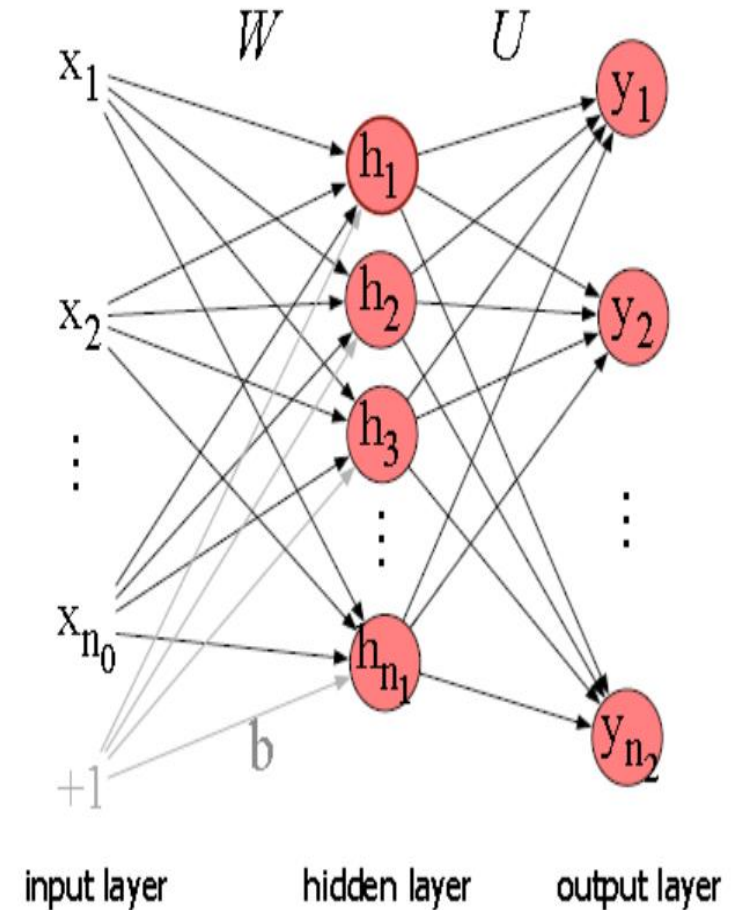
- multiplying the weight matrix by the input vector x ,
- adding the bias vector b ,
- and applying the activation function g (**such as the sigmoid, tanh, or ReLU activation**)
- $h = \sigma(Wx + b)$
- **sigmoid function σ** as our activation function

Feedforward Neural Networks

- input layer as layer 0 of the network, and have n_0 represent the number of inputs, so x is a vector of real numbers of dimension n_0 ,
- or more formally $x \in \mathbb{R}^{n_0}$, a column vector of dimensionality $[n_0; 1]$.
- The hidden layer has dimensionality n_1 , so $h \in \mathbb{R}^{n_1}$ and also $b \in \mathbb{R}^{n_1}$
- (since each hidden unit can take a different bias value).
- And the **weight matrix W** has dimensionality $W \in \mathbb{R}^{n_1 \times n_0}$ i.e. **$[n_1; n_0]$** .
- matrix multiplication will compute the value of each h_j as
- $$h = \sigma\left(\sum_{i=0}^{n_0} W_{ji} x_i + b_j\right)$$

Feedforward Neural Networks

- Like the hidden layer, the output layer has a weight matrix (let's call it U), but some models don't include a bias vector b in the output layer
- $Z = Uh$
- There are n_2 output nodes, so $z \in \mathbb{R}^{n_2}$, weight matrix U has dimensionality $U \in \mathbb{R}^{n_2 \times n_1}$
- element U_{ij} is the weight from unit j in the hidden layer to unit i in the output layer.



Feedforward Neural Networks

- $\mathbf{Z} = \mathbf{U}\mathbf{h}$
- However, $\mathbf{z}$ can't be the output of the classifier, since it's a vector of real-valued numbers, while what we need for classification is a vector of **probabilities**.
- convenient function for normalizing a vector of real values, by which we mean converting it to a vector that encodes a probability distribution(all numbers lie in between 0 and 1 and sum to 1) the softmax function.

$$\text{softmax}(\mathbf{z}_i) = \frac{\exp(\mathbf{z}_i)}{\sum_{j=1}^d \exp(\mathbf{z}_j)} \quad 1 \leq i \leq d$$

Feedforward Neural Networks

$$\text{softmax}(\mathbf{z}_i) = \frac{\exp(\mathbf{z}_i)}{\sum_{j=1}^d \exp(\mathbf{z}_j)} \quad 1 \leq i \leq d$$

Thus for example given a vector

$$\mathbf{z} = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1],$$

the softmax function will normalize it to a probability distribution (s

$$\text{softmax}(\mathbf{z}) = [0.055, 0.090, 0.0067, 0.10, 0.74, 0.010]$$

we used softmax to create a probability distribution from a vector of real-valued numbers

Feedforward Neural Networks

- So a neural network is like **multinomial logistic regression**, but
- (a) with many layers, since a deep neural network is like layer after layer of **logistic regression classifiers**;
- (b) with those intermediate layers having many possible activation functions (**tanh, ReLU, sigmoid**) instead of just sigmoid

Feedforward Neural Networks

- Here are the final equations for a feedforward network with a single hidden layer, which takes an input vector x , outputs a probability distribution y , and is parameterized by weight matrices W and U and a bias vector b :
- $h = \sigma(Wx + b)$
- $z = Uh$
- $y = \text{softmax}(z)$
- $x \in \mathbb{R}^{n_0}$, $h \in \mathbb{R}^{n_1}$, $b \in \mathbb{R}^{n_1}$
- $W \in \mathbb{R}^{n_1 \times n_0}$, $U \in \mathbb{R}^{n_2 \times n_1}$
- $y \in \mathbb{R}^{n_2}$
- So by this terminology logistic regression is a 1-layer network.

More details on feedforward networks

- deeper networks of depth more than 2.
- starting at 0 for the input layer.
- So $W^{[1]}$ will mean the weight matrix for the (first) hidden layer,
- and $b^{[1]}$ will mean the bias vector for the (first) hidden layer.
- n_j will mean the number of units at layer j .
- We'll use $g(\cdot)$ to stand for the activation function,
- which will tend to be ReLU or tanh for intermediate layers and softmax for output layers.

More details on feedforward networks

- use $a^{[i]}$ to mean the **output from layer i**,
- $z^{[i]}$ to mean the combination of previous layer output, weights and biases
- $\mathbf{W}^{[i]} \mathbf{a}^{[i-1]} + \mathbf{b}^{[i]}$
- The 0th layer is for inputs, refer to the inputs x more generally as $a^{[0]}$.
- Thus we can re-represent our 2-layer net as follows
- $z^{[1]} = W^{[1]} a^{[0]} + b^{[1]}$
- $a^{[1]} = g^{[1]} z^{[1]}$
- $z^{[2]} = W^{[2]} a^{[1]} + b^{[2]}$
- $a^{[2]} = g^{[2]} z^{[2]}$
- $\hat{y} = a^{[2]}$

More details on feedforward networks

- The algorithm for computing the forward step in an n-layer feedforward network, given the input vector $a^{[0]}$ is thus simply:
- for i in $1, \dots, n$

$$z^{[i]} = W^{[i]} a^{[i-1]} + b^{[i]}$$

$$a^{[i]} = g^{[i]} z^{[i]}$$

$$\hat{y} = a^{[n]}$$

More details on feedforward networks

- One of the reasons we use **nonlinear activation functions** for each layer in a neural network is that if we did not, the resulting network is exactly equivalent to a single-layer network. Why?
- Imagine the first two layers of such a network of purely linear layers:

$$\begin{aligned}\mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]} \\ \mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{z}^{[1]} + \mathbf{b}^{[2]}\end{aligned}$$

We can rewrite the function that the network is computing as:

$$\begin{aligned}\mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{z}^{[1]} + \mathbf{b}^{[2]} \\ &= \mathbf{W}^{[2]}(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}) + \mathbf{b}^{[2]} \\ &= \mathbf{W}^{[2]}\mathbf{W}^{[1]}\mathbf{x} + \mathbf{W}^{[2]}\mathbf{b}^{[1]} + \mathbf{b}^{[2]} \\ &= \mathbf{W}'\mathbf{x} + \mathbf{b}'\end{aligned}$$

More details on feedforward networks

- This generalizes to any number of layers.
- So without non-linear activation functions, a multilayer network is just a notational variant of a single layer network with a different set of weights,
- and we lose all the representational power of multilayer networks.
- **Replacing the bias unit**
- we add a dummy node a_0 to each layer whose value will always be 1.
layer 0, the input layer, will have a dummy node $a_0^{[0]} = 1$
- layer 1 will have $a_0^{[1]} = 1$, For example instead of an equation like
- $h = \sigma(Wx + b)$
- We will use $h = \sigma(Wx)$

More details on feedforward networks

But now instead of our vector $\mathbf{x}$ having n_0 values: $\mathbf{x} = \mathbf{x}_1, \dots, \mathbf{x}_{n_0}$, it will have $n_0 + 1$ values, with a new 0th dummy value $\mathbf{x}_0 = 1$: $\mathbf{x} = \mathbf{x}_0, \dots, \mathbf{x}_{n_0}$. And instead of computing each $\mathbf{h}_j$ as follows:

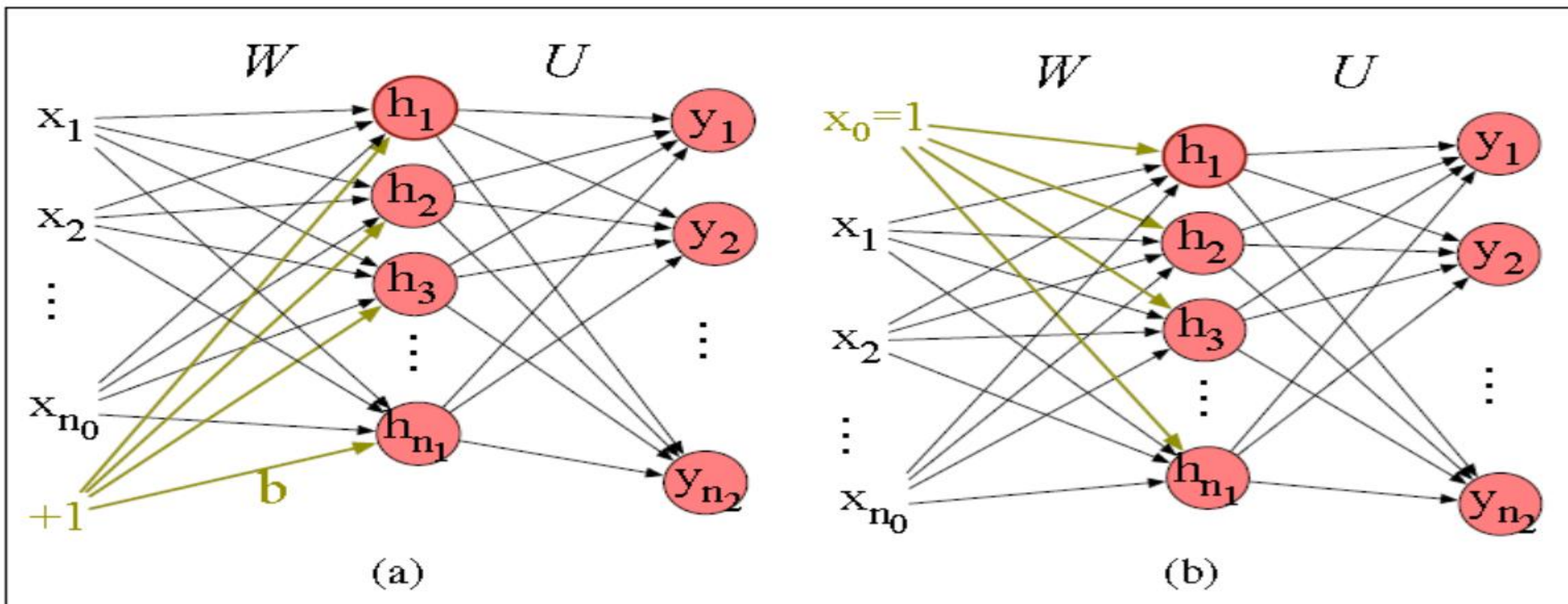
$$\mathbf{h}_j = \sigma \left(\sum_{i=1}^{n_0} \mathbf{w}_{ji} \mathbf{x}_i + \mathbf{b}_j \right),$$

we'll instead use:

$$\mathbf{h}_j = \sigma \left(\sum_{i=0}^{n_0} \mathbf{w}_{ji} \mathbf{x}_i \right),$$

where the value $\mathbf{w}_{j0}$ replaces what had been $\mathbf{b}_j$.

More details on feedforward networks



Replacing the bias node (shown in a) with x_0 (b).

Training Neural Network

- The goal of the training procedure is to learn parameters $W^{[i]}$ and $b^{[i]}$ for each layer i that make $\hat{y}$ for each training observation as close as possible to the true y .
- **First**, we'll need a loss function that models the distance between the system output and the gold output,
- loss function used for logistic regression, the **cross-entropy loss**.
- Second, to find the parameters that **minimize this loss function**,
- use the **gradient descent optimization algorithm**.
- **Third, gradient descent** requires knowing the gradient of the loss function,
- the vector that contains the partial derivative of the loss function with respect to each of the parameters.

Training Neural Network

- In logistic regression, for each observation we could directly compute the derivative of the loss function with respect to an individual w or b .
- But for neural networks, with millions of parameters in many layers,
- it's much harder to see how to compute the partial derivative of some weight in layer 1 when the loss is attached to some much later layer.
- How do we partial out the loss over all those intermediate layers?
- The answer is the algorithm called **error backpropagation or backward differentiation**.

Training Neural Network - Loss function

- If the neural network is being used as a binary classifier, with sigmoid at the final layer, the loss function is the same logistic regression loss.

$$L_{\text{CE}}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

$$= -[y \log \sigma(w \cdot x + b) + (1 - y) \log(1 - \sigma(w \cdot x + b))]$$

- If we are using the network to classify into 3 or more classes, the loss function is exactly the same as the loss for multinomial regression

Loss Function

- when we have more than 2 classes we'll need to represent both y and $\hat{y}$ as vectors.
- Let's assume we're doing hard classification, where only one class is the correct one.
- true label y is then a vector with K elements, each corresponding to a class, with $y_c = 1$ if the correct class is c , with all other elements of y being 0.
- with one value equal to 1 and the rest 0, is called a **one-hot vector**.
- And our classifier will produce an estimate vector with K elements $\hat{y}$,
- each element $\hat{y}_k$ of which represents the estimated probability **$p(y_k = 1 \mid x)$** .

Loss Function

- The loss function for a single example x is the negative sum of the logs of the K output classes, each weighted by their probability y_k :

$$L_{CE}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{k=1}^K y_k \log \hat{y}_k$$

- let's first rewrite the equation using the function $\mathbf{1}\{\}$ which evaluates to 1 if the condition in the brackets is true and to 0 otherwise.
- $L_{CE}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{k=1}^K \mathbf{1}\{y_k = 1\} \log \hat{y}_k$

Loss Function

- the cross-entropy loss is simply the negative log of the output probability
- corresponding to the correct class, and we therefore also call this the negative.
- $L_{CE}(\hat{\mathbf{y}}, \mathbf{y}) = -\log \hat{y}_c$ (where c is the correct class)
- Plugging in the softmax formula and with K the number of classes.

$$L_{CE}(\hat{\mathbf{y}}, \mathbf{y}) = -\log \frac{\exp(\mathbf{z}_c)}{\sum_{j=1}^K \exp(\mathbf{z}_j)} \quad (\text{where } c \text{ is the correct class})$$

Computing the Gradient

- Computing the gradient requires the partial derivative of the loss function with respect to each parameter.
- use the derivative of the loss that we used for logistic regression

$$\begin{aligned}\frac{\partial L_{CE}(\hat{\mathbf{y}}, \mathbf{y})}{\partial w_j} &= (\hat{y} - y) \mathbf{x}_j \\ &= (\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y) \mathbf{x}_j\end{aligned}$$

Computing the Gradient

- for a network with one weight layer and softmax output (=multinomial logistic regression),
- we could use the **derivative of the softmax loss** , shown for a particular weight $\mathbf{w}_k$ and input $\mathbf{x}_i$.

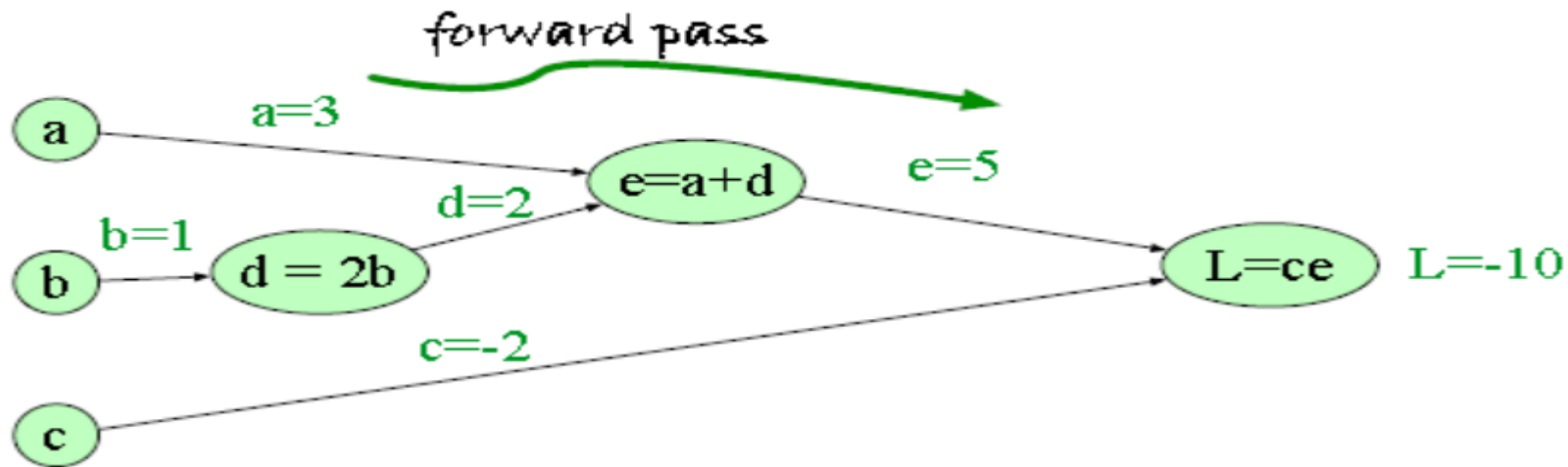
$$\begin{aligned}\frac{\partial L_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y})}{\partial \mathbf{w}_{k,i}} &= -(\mathbf{y}_k - \hat{\mathbf{y}}_k) \mathbf{x}_i \\ &= -(\mathbf{y}_k - p(\mathbf{y}_k = 1 | \mathbf{x})) \mathbf{x}_i \\ &= -\left(\mathbf{y}_k - \frac{\exp(\mathbf{w}_k \cdot \mathbf{x} + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)} \right) \mathbf{x}_i\end{aligned}$$

Computing the Gradient

- For deep networks, computing gradients for each weight is much complex,
- since we are computing the derivative with respect to weight parameters that appear all the way back in the very early layers of the network,
- even though the loss is computed only at the very end of the network.
- The solution to computing this gradient is an algorithm called error backpropagation.
- it turns out to be the same as a more general procedure called backward differentiation, which depends on the notion of **computation graphs**.

Computation Graphs

- Consider computing the function $L(a,b,c) = c(a+2b)$.
- $d = 2 * b$, $e = a+d$, $L = c * e$
- In the figure, we've assumed the inputs $a = 3$, $b = 1$, $c = -2$,
- the result of the forward pass to compute the result **$L(3,1,-2) = -10$**



Backward differentiation on computation graphs

- goal is to compute the derivative of the output function L with respect to each of the input variables

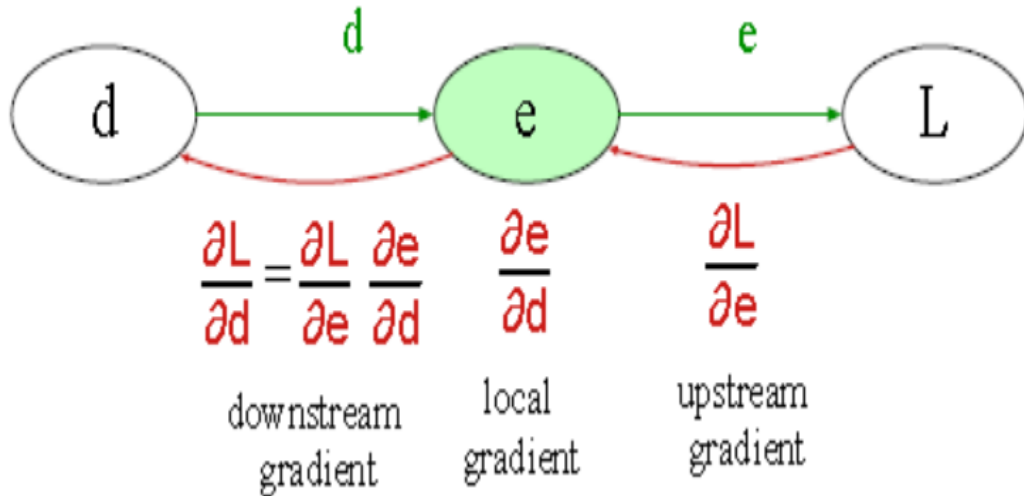
- $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, *and* $\frac{\partial L}{\partial c}$,

- $\frac{\partial L}{\partial a}$ tells us how much a small change in a affects L .

- chain rule

- $f(x) = u(v(x))$

- $\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$

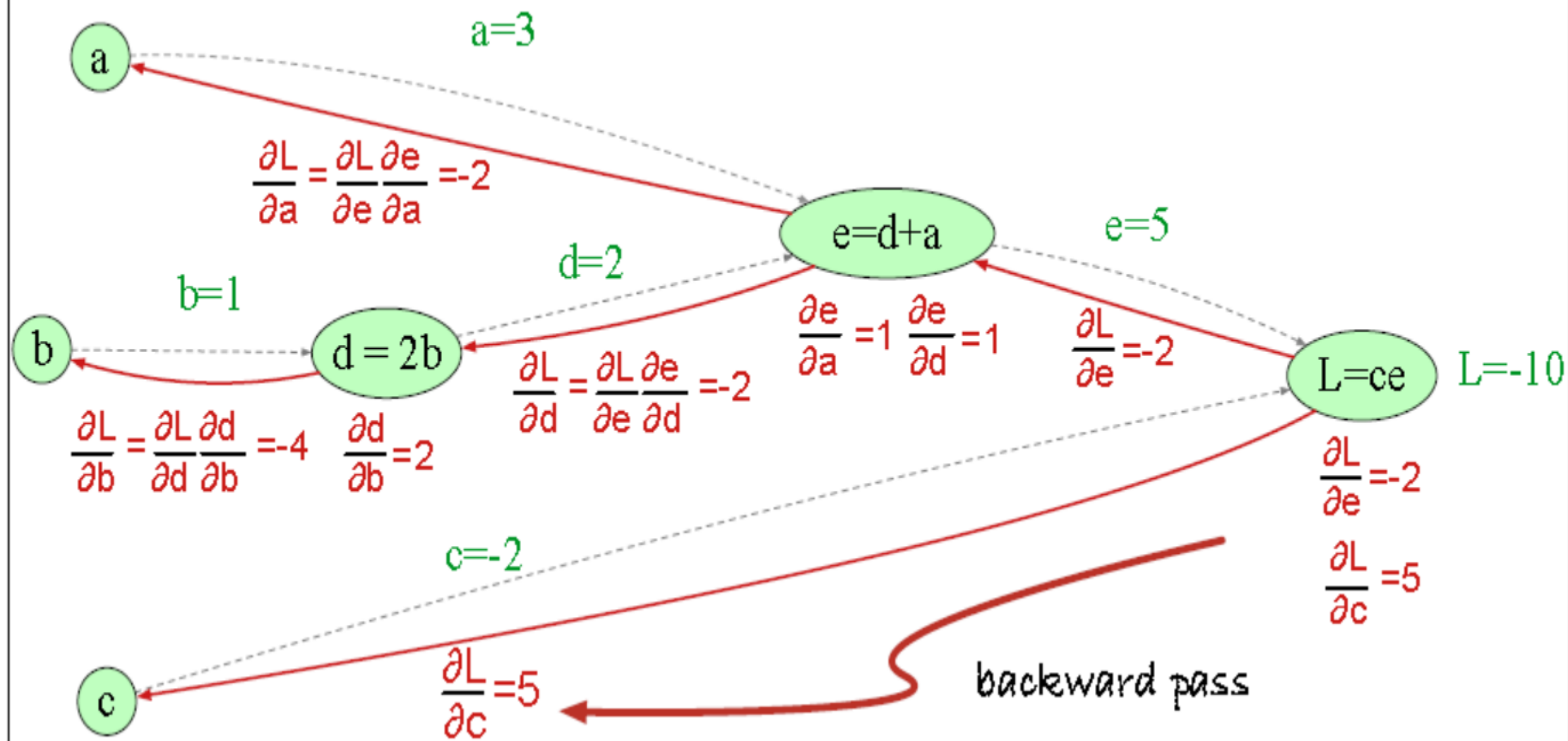


- Let's now compute the 3 derivatives we need.
- Since in the computation graph
- $L = ce$,
- $\frac{\partial L}{\partial c} = e$
- For the other two, we'll need to use the chain rule:
- $\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial a}$
- $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial d} \cdot \frac{\partial d}{\partial b}$

$$L = ce : \quad \frac{\partial L}{\partial e} = c, \frac{\partial L}{\partial c} = e$$

$$e = a + d : \quad \frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial d} = 1$$

$$d = 2b : \quad \frac{\partial d}{\partial b} = 2$$



Backward differentiation for a neural network

- Fig. 7.15 shows a sample computation graph for a 2-layer neural network with $n_0 = 2$, $n_1 = 2$, and $n_2 = 1$, assuming binary classification and hence using a sigmoid output unit for simplicity.

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$$

$$\mathbf{a}^{[1]} = \text{ReLU}(\mathbf{z}^{[1]})$$

$$z^{[2]} = \mathbf{W}^{[2]}\mathbf{a}^{[1]} + b^{[2]}$$

$$a^{[2]} = \sigma(z^{[2]})$$

$$\hat{y} = a^{[2]}$$

Backward differentiation for a neural network

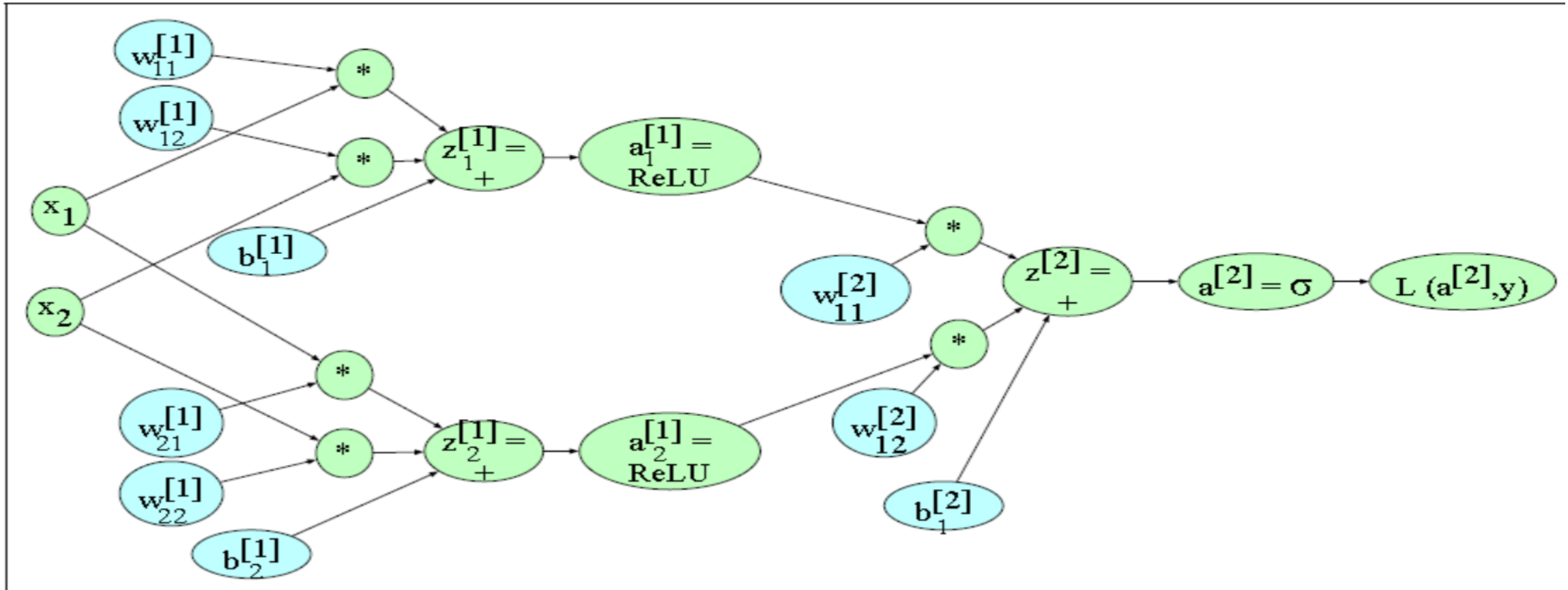
- For the backward pass we'll also need to compute the loss L . The loss function for binary sigmoid output from Eq. 7.23 is

$$L_{CE}(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

Our output $\hat{y} = a^{[2]}$, so we can rephrase this as

$$L_{CE}(a^{[2]}, y) = -[y \log a^{[2]} + (1 - y) \log(1 - a^{[2]})]$$

Backward differentiation for a neural network



Sample computation graph for a simple 2-layer neural net (= 1 hidden layer) with two input units and 2 hidden units.

Backward differentiation for a neural network

- the derivative of the sigmoid σ : $\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$
- We'll also need the derivatives of each of the other activation functions.

- The derivative of tanh is: $\frac{d \tanh(z)}{dz} = 1 - \tanh^2(z)$

The derivative of the ReLU is²

$$\frac{d\text{ReLU}(z)}{dz} = \begin{cases} 0 & \text{for } z < 0 \\ 1 & \text{for } z \geq 0 \end{cases}$$

Backward differentiation for a neural network

- computing the derivative of loss function L with respect to z , or $\frac{\partial L}{\partial a}$
- $\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a^2} \cdot \frac{\partial a^2}{\partial z}$

$$\begin{aligned} L_{CE}(a^{[2]}, y) &= - \left[y \log a^{[2]} + (1 - y) \log(1 - a^{[2]}) \right] \\ \frac{\partial L}{\partial a^{[2]}} &= - \left(\left(y \frac{\partial \log(a^{[2]})}{\partial a^{[2]}} \right) + (1 - y) \frac{\partial \log(1 - a^{[2]})}{\partial a^{[2]}} \right) \\ &= - \left(\left(y \frac{1}{a^{[2]}} \right) + (1 - y) \frac{1}{1 - a^{[2]}} (-1) \right) \\ &= - \left(\frac{y}{a^{[2]}} + \frac{y - 1}{1 - a^{[2]}} \right) \end{aligned}$$

Backward differentiation for a neural network

Next, by the derivative of the sigmoid:

$$\frac{\partial a^{[2]}}{\partial z} = a^{[2]}(1 - a^{[2]})$$

Finally, we can use the chain rule:

$$\begin{aligned}\frac{\partial L}{\partial z} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial z} \\ &= - \left(\frac{y}{a^{[2]}} + \frac{y-1}{1-a^{[2]}} \right) a^{[2]}(1-a^{[2]}) \\ &= a^{[2]} - y\end{aligned}$$

More details on learning

- Optimization in neural networks is a non-convex optimization problem, more complex than for logistic regression.
- For logistic regression we can initialize gradient descent with all the weights and biases having the value 0.
- In neural networks, by contrast, we need to initialize the weights with small random numbers.
- It's also helpful to normalize the input values to have 0 mean and unit variance.

More details on learning

- **Various forms of regularization are used to prevent overfitting.**
- most important is **dropout**: randomly dropping some units and their connections from the network during training.
- Tuning of hyperparameters is also important.
- The parameters of a neural network are the weights W and biases b ; those are learned by gradient descent.
- Hyperparameters include the learning rate h , the mini-batch size, the model architecture (number of layers, the number of hidden nodes per layer, choice of activation functions), how to regularize, and so on.

More details on learning

- Gradient descent itself also has many architectural variants such as Adam
- Finally, most modern neural networks are built using computation graph formalisms that make it easy and natural to do gradient computation and parallelization on vector-based GPUs (Graphic Processing Units).
- **PyTorch and TensorFlow** are two of the most popular

Deep Feedforward Networks

- Deep Feedforward Networks
- or feedforward neural networks
- or multilayer perceptrons (MLPs),
- Goal of a feedforward network is to approximate some function f^*
- For example,
- for a classifier, $y = f^*(x)$ maps an input x to a category y .
- A feedforward network defines a mapping $y = f(x; \theta)$
- and learns the value of the parameters θ that result in the best function approximation.

Deep Feedforward Networks

- These models are called feedforward because information flows through the function being evaluated from x , through the intermediate computations used to define f , and finally to the output y .
- There are no feedback connections in which outputs of the model are fed back into itself.
- When feedforward neural networks are extended to include feedback connections, they are called **recurrent neural networks**,

Deep Feedforward Networks

- Each hidden layer of the network is typically vector-valued.
- dimensionality of these hidden layers determines the width of the model.
- Each element of the vector may be interpreted analogous to a neuron.
- layer as consisting of many units.
- Each unit resembles a neuron
- feedforward networks is to begin with linear models Linear models, such as logistic regression and linear regression, are appealing because they may be fit efficiently and reliably, either in closed form or with convex optimization

Preceptron Network

- **a linear classifier** it can classify an input into one of two possible outputs.
- A simple perceptron is a single layer feed-forward NN that uses a supervised learning rule for training.
- the initial perceptron has three layers in its architecture as **sensory neurons, associator neurons, and a response neuron** which is sometimes considered to be analogous of a **retina model**.
- weights with value +1, 0, or -1 assigned at random.
- The transfer function for each associator units is a binary step function with arbitrary fixed threshold.
- signal sent from the associator units to the output unit is either 1 or 0.

Preceptron Network

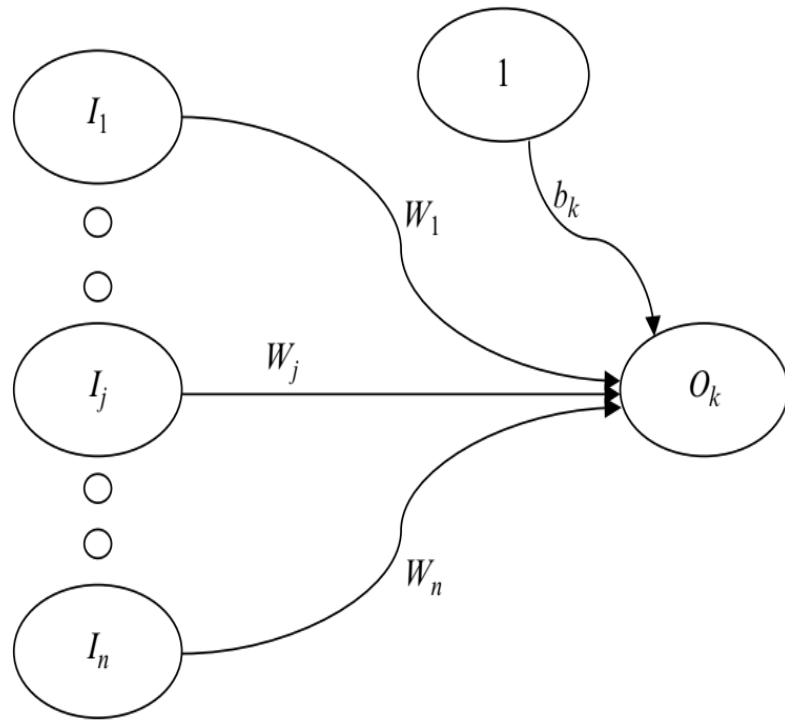
- transfer function used in the output neuron of the perceptron

$$\bullet f(\text{Sum}_k) = \begin{cases} 1 & \text{if } \text{Sum}_k > \theta_k \\ 0 & \text{if } -\theta_k \leq \text{Sum}_k \leq \theta_k \\ -1 & \text{if } \text{Sum}_k < -\theta_k \end{cases}$$

- $\theta_k = 0.5$, $W_1=0$, $W_2=1$, $n=1$
- Then the weights will be updated

$$W_j(t) = W_j(t-1) + \eta \times D_k \times I_j$$

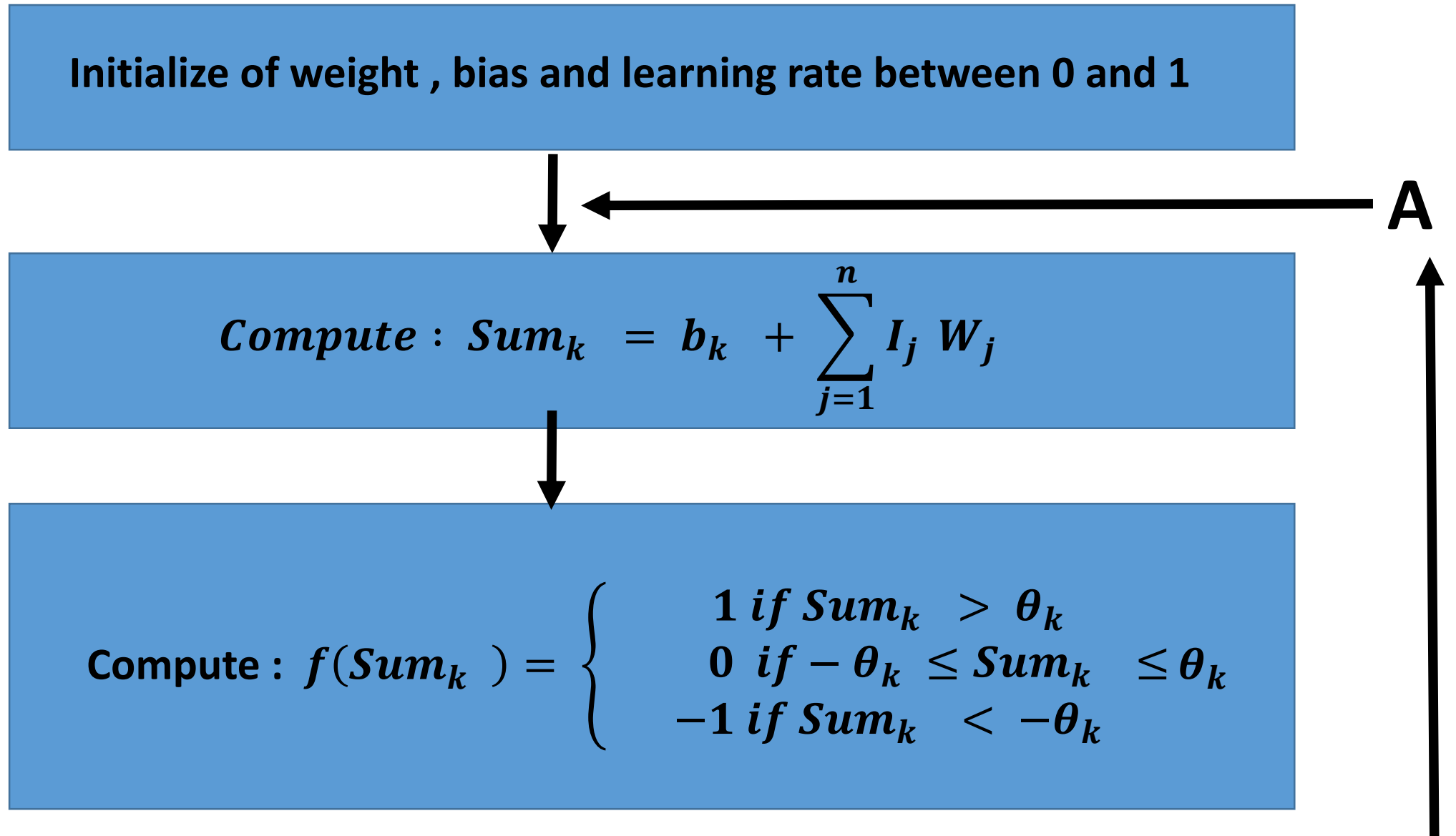
Preceptron Network

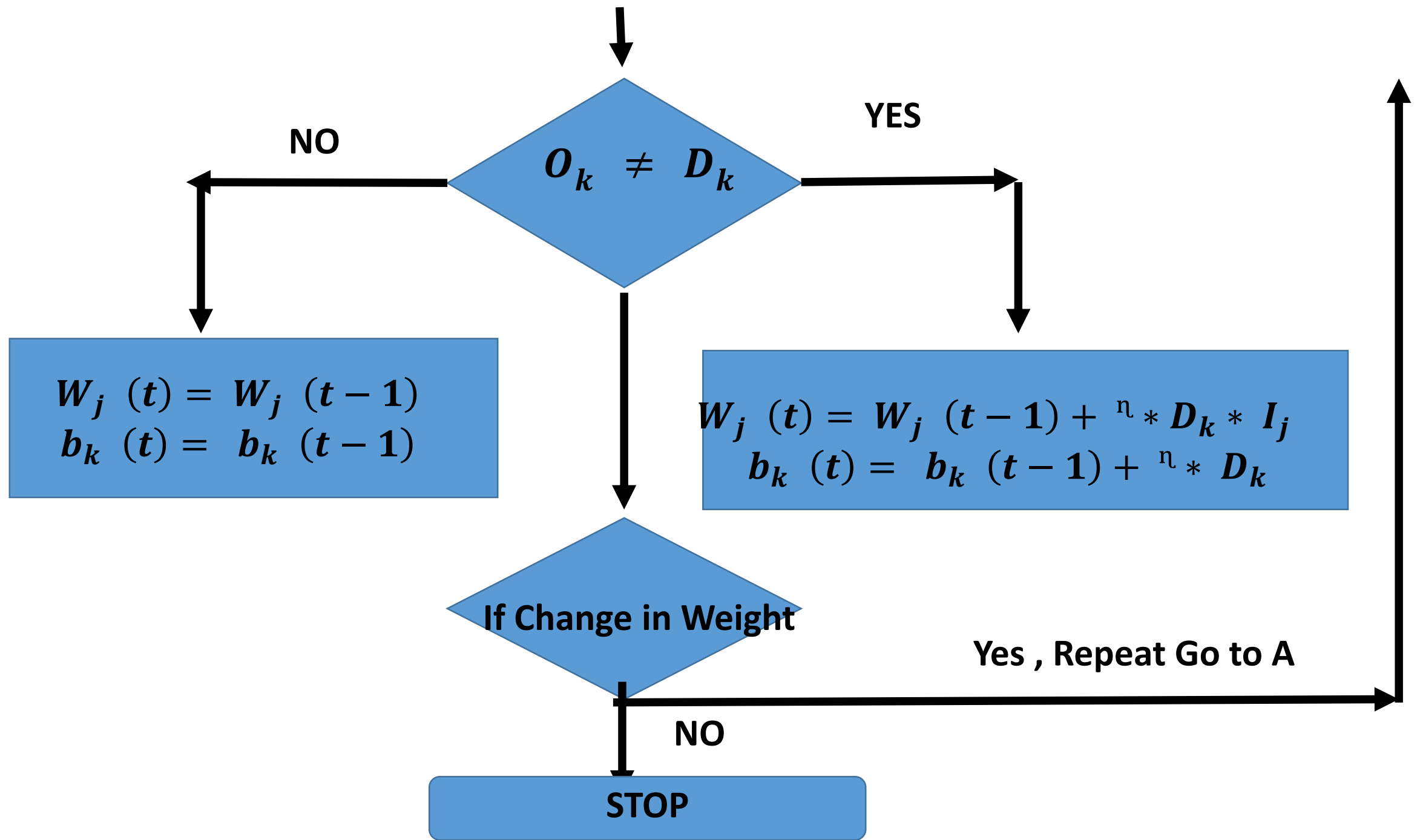


AND	True(1)	False(0)
True (1)	True	False
False(0)	False	False

Input (And)	Desire Output
(1, 1)	1
(1, 0)	-1
(0, 1)	-1
(0, 0)	-1

Preceptron Network





$$W_j(t) = W_j(t-1)$$

$$b_k(t) = b_k(t-1)$$

$$W_j(t) = W_j(t-1) + \eta * D_k * I_j$$

$$b_k(t) = b_k(t-1) + \eta * D_k$$

Input		$Sum_k = 1$	$O_{k=1}$	$D_k = 1$	Weight(W)		Bias
I_1	I_2	$= b_k + \sum_{j=1}^n I_j W_j$	Output	Desired	$W_1 = 0$	$W_2 = 1$	$b_k = 0$
1	1	$= 0 + 1 * 0 + 1 * 1$ $= 1$	1	1	Since $O_k = D_k$ 0	1	0
1	0	$= 0 + 1 * 0 + 0 * 1$ $= 1$	1	-1	$= 0 + 1 * (-1) * 1$ $= -1$	$= 1 + 1 * (-1) * 0$ $= 1$	$= 0 + 1 * -1 = -1$ -1
0	1	$= -1 + 0 * -1 + 1 * 1$ $= 0$	0	-1	$= -1 + 1 * (-1) * 0$ $= -1$	$= 1 + 1 * (-1) * 1$ $= 0$	$= -1 + 1 * -1 = -1$ -2
0	0	$= -2 + 0 * -1 + 0 * 1$ $= -2$	-1	-1	Since $O_k = D_k$ -1	0	-2



$$W_j(t) = W_j(t-1)$$

$$b_k(t) = b_k(t-1)$$



$$W_j(t) = W_j(t-1) + \eta * D_k * I_j$$

$$b_k(t) = b_k(t-1) + \eta * D_k$$

1	1	$= -2 + 1 * -1 + 1 * 0$ $= -3$	-1	1	$= -1 + 1 * (1) * 1$ $= 0$	$= 0 + 1 * (1) * 1$ $= 1$	$= -2 + 1 * 1 = -1$ -1
1	0	$= -1 + 1 * 0 + 0 * 1$ $= -1$	-1	-1	Since $O_k = D_k$ <u>0</u>	1	-1
0	1	$= -1 + 0 * 0 + 1 * 1$ $= 0$	0	-1	$= 0 + 1 * (-1) * 0$ $= 0$	$= 1 + 1 * (-1) * 1$ $= 0$	$= -1 + 1 * -1 = -2$ -2
0	0	$= -2 + 0 * 0 + 0 * 0$ $= -2$	-1	-1	Since $O_k = D_k$ <u>0</u>	0	-2

training process for AND logic function

0	0	$=-2+0*1+0*1$ $=-2$	-1	-1	Since $O_k = D_k$ -1	0	-2
1	1	$=-2+1*-1+1*0$ $=-3$	-1	1	$=-1+1*(1)*1$ $=0$	$=0+1*(1)*1$ $=1$	$=-2+1*1=-1$ -1
1	0	$=-1+1*0+0*1$ $=-1$	-1	-1	Since $O_k = D_k$ 0	1	-1
0	1	$=-1+0*0+1*1$ $=0$	0	-1	$=0+1*(-1)*0$ $=0$	$=1+1*(-1)*1$ $=0$	$=-1+1*-1=-2$ -2
0	0	$=-2+0*0+0*0$ $=-2$	-1	-1	Since $O_k = D_k$ 0	0	-2

Full Steps -training process for AND logic



Input		$Sum_k = 1$	$O_{k=1}$	$D_k = 1$	Weight(W)		Bias
I_1	I_2	$= b_k + \sum_{j=1}^n I_j W_j$	Output	Desired	$W_1 = 0$	$W_2 = 1$	$b_k = 0$
1	1	1	1	1	0	1	0
1	0	0	0	-1	-1	1	-1
0	1	0	0	-1	-1	0	-2
0	0	-2	-1	-1	-1	0	-2
1	1	-3	-1	1	0	1	-1
1	0	-1	-1	-1	0	1	-1
0	1	0	0	-1	0	0	-2
0	0	-2	-1	-1	0	0	-2

Full Steps -training process for AND logic

Input		$Sum_k = 1$	$O_{k=1}$	$D_k = 1$	Weight(W)		Bias
I_1	I_2	$= b_k + \sum_{i=1}^n I_i W_i$	Output	Desired	$W_1 = 0$	$W_2 = 1$	$b_k = 0$
1	1	-2	-1	1	1	1	-1
1	0	0	0	-1	0	1	-1
0	1	-1	-1	-1	0	1	-2
0	0	-2	-1	-1	0	1	-2
1	1	-1	-1	1	1	2	-1
1	0	0	0	-1	0	2	-2
0	1	0	0	-1	0	1	-3
0	0	3	-1	-1	0	1	-3

Full Steps -training process for AND logic

Input		$Sum_k = 1$	$O_{k=1}$	$D_k = 1$	Weight(W)		Bias
I_1	I_2	$= b_k + \sum_{j=1}^n I_j W_j$	Output	Desired	$W_1 = 0$	$W_2 = 1$	$b_k = 0$
1	1	-2	-1	1	1	2	-2
1	0	-1	-1	-1	1	2	-2
0	1	0	0	-1	1	1	-3
0	0	-3	-1	-1	1	1	-3
1	1	-1	-1	1	2	2	-2
1	0	0	0	-1	1	2	-3
0	1	-1	-1	-1	1	2	-3
0	0	-3	-1	-1	1	2	-3

Full Steps -training process for AND logic

Input		$Sum_k = 1$	$O_{k=1}$	$D_k = 1$	Weight(W)		Bias
I_1	I_2	$= b_k + \sum_{j=1}^n I_j W_j$	Output	Desired	$W_1 = 0$	$W_2 = 1$	$b_k = 0$
1	1	0	0	1	2	3	-2
1	0	0	0	-1	1	3	-3
0	1	0	0	-1	1	2	-4
0	0	-4	-1	-1	1	2	-4
1	1	-1	-1	1	2	3	-3
1	0	-1	-1	-1	2	3	-3
0	1	0	0	-1	0	0	-4
0	0	-4	-1	-1	2	2	-4

Full Steps -training process for AND logic

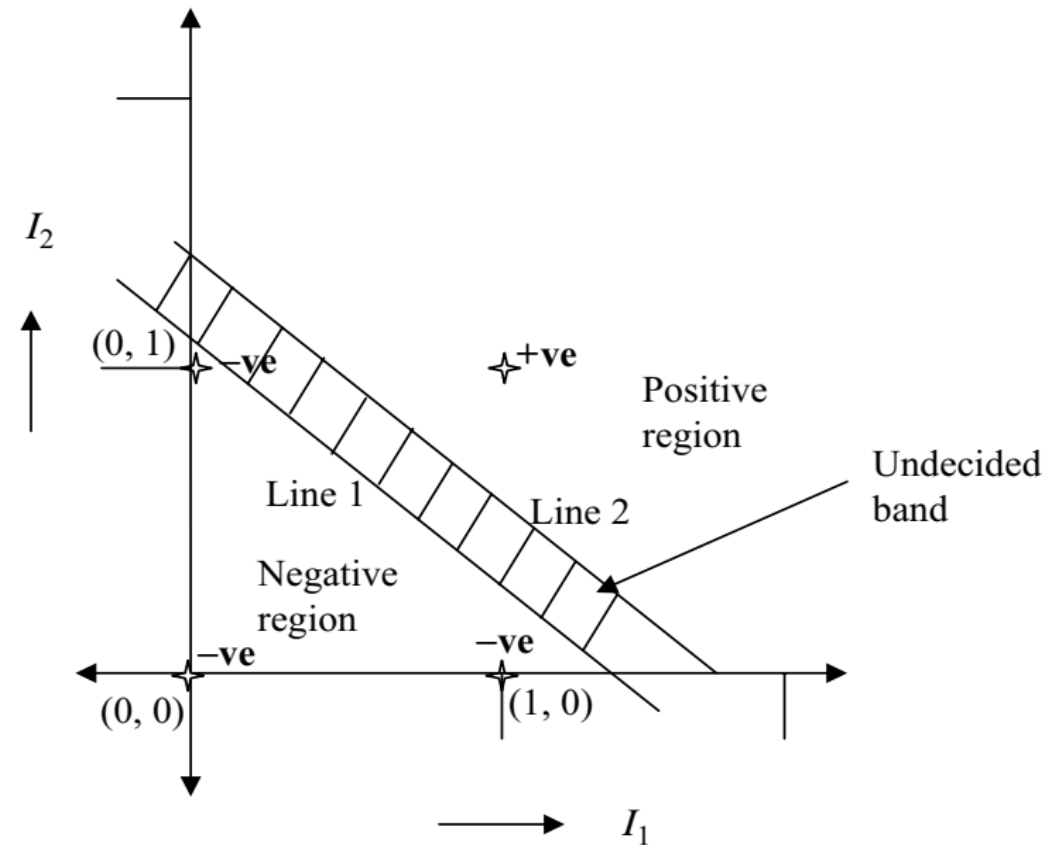
Input		$Sum_k = 1$	$O_{k=1}$	$D_k = 1$	Weight(W)		Bias
I_1	I_2	$= b_k + \sum_{j=1}^n I_j W_j$	Output	Desired	$W_1 = 0$	$W_2 = 1$	$b_k = 0$
1	1	0	0	1	3	3	-3
1	0	0	0	-1	2	3	-4
0	1	-1	-1	-1	2	3	-4
0	0	-4	-1	-1	2	3	-4
1	1	1	1	1	2	3	-4
1	0	-2	-1	-1	2	3	-4
0	1	-1	-1	-1	2	3	-4
0	0	-4	-1	-1	2	3	-4

Full Steps -training process for AND logic

- The final weights and biases after training
- is found to $W_1 = 2$, $W_2 = 3$, and $b_k = -4$.
- Two line which forms the undecided band.
- This can be plotted in a two-dimensional plot for two inputs I_1 I_2
- $W_1 * I_1 + W_2 * I_2 + b_k > \theta_k$
- $W_1 * I_1 + W_2 * I_2 + b_k < -\theta_k$
- $2 * I_1 + 3 * I_2 - 4 > 0.5$ (Positive region)
- $2 * I_1 + 3 * I_2 - 4 < -0.5$ (Negative Region)
- Undecided Region
- $-\theta_k \leq Sum_k \leq \theta_k$

- $2 * I_1 + 3 * I_2 - 4 > 0.5$ (Positive region)
- $2 * I_1 + 3 * I_2 - 4 < -0.5$ (Negative Region)

	$I_2 = 0$	$I_2 = 1$
$I_1 = 0$	$-4 < 0.5$ -(negative)	$-1 < 0.5$ -(negative)
$I_1 = 1$	$-2 < 0.5$ - (Negative)	$1 > 0.5$ +Positive



OR function

Similarly, the computation of step-by-step training process is carried out for OR function and the results are presented

Input (I)		$sum_{k=1}$	$O_{k=1}$ Output response	$D_{k=1}$ Desired target	Weights (W)		Bias
I_1	I_2				$W_1 = 1$	$W_2 = 0$	$b_k = 0$
1	1	1	1	1	0	0	0
1	0	0	0	1	0	0	0
0	1	2	1	1	1	1	1
0	0	1	1	-1	1	1	0
1	1	2	1	1	1	1	0
1	0	1	1	1	1	1	0

OR function

Input (I)		$sum_{k=1}$	$O_{k=1}$ Output response	$D_{k=1}$ Desired target	Weights (W)		Bias
I_1	I_2				$W_1 = 1$	$W_2 = 0$	$b_k = 0$
0	1	1	1	1	1	1	0
0	0	0	0	-1	1	1	-1
1	1	1	1	1	1	1	-1
1	0	0	0	1	2	2	0
0	1	1	1	1	2	1	0
0	0	0	0	-1	2	1	-1
1	1	2	1	1	2	1	-1
1	0	1	1	1	2	1	-1
0	1	0	0	1	2	2	0
0	0	0	0	-1	2	2	-1
1	1	3	1	1	2	2	-1
1	0	1	1	1	2	2	-1
0	1	1	1	1	2	2	-1
0	0	-1	-1	-1	2	2	-1

OR function

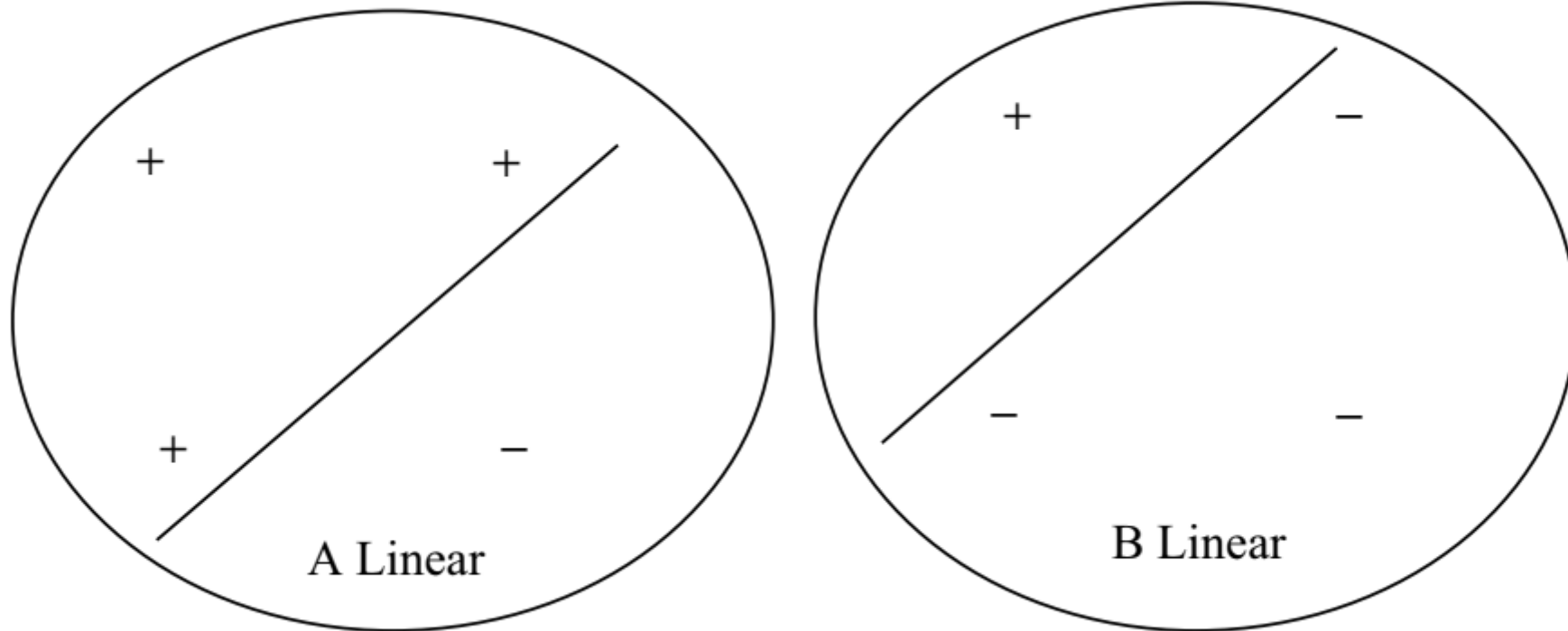
1	1	3	1	1	2	2	-1
1	0	1	1	1	2	2	-1
0	1	1	1	1	2	2	-1
0	0	-1	-1	-1	2	2	-1
1	1	3	1	1	2	2	-1
1	0	1	1	1	2	2	-1
0	1	1	1	1	2	2	-1
0	0	-1	-1	-1	2	2	-1
1	1	3	1	1	2	2	-1
1	0	1	1	1	2	2	-1
0	1	1	1	1	2	2	-1
0	0	-1	-1	-1	2	2	-1
1	1	3	1	1	2	2	-1
1	0	1	1	1	2	2	-1
0	1	1	1	1	2	2	-1
0	0	-1	-1	-1	2	2	-1

OR function

1	1	3	1	1	2	2	-1
1	0	1	1	1	2	2	-1
0	1	1	1	1	2	2	-1
0	0	-1	-1	-1	2	2	-1

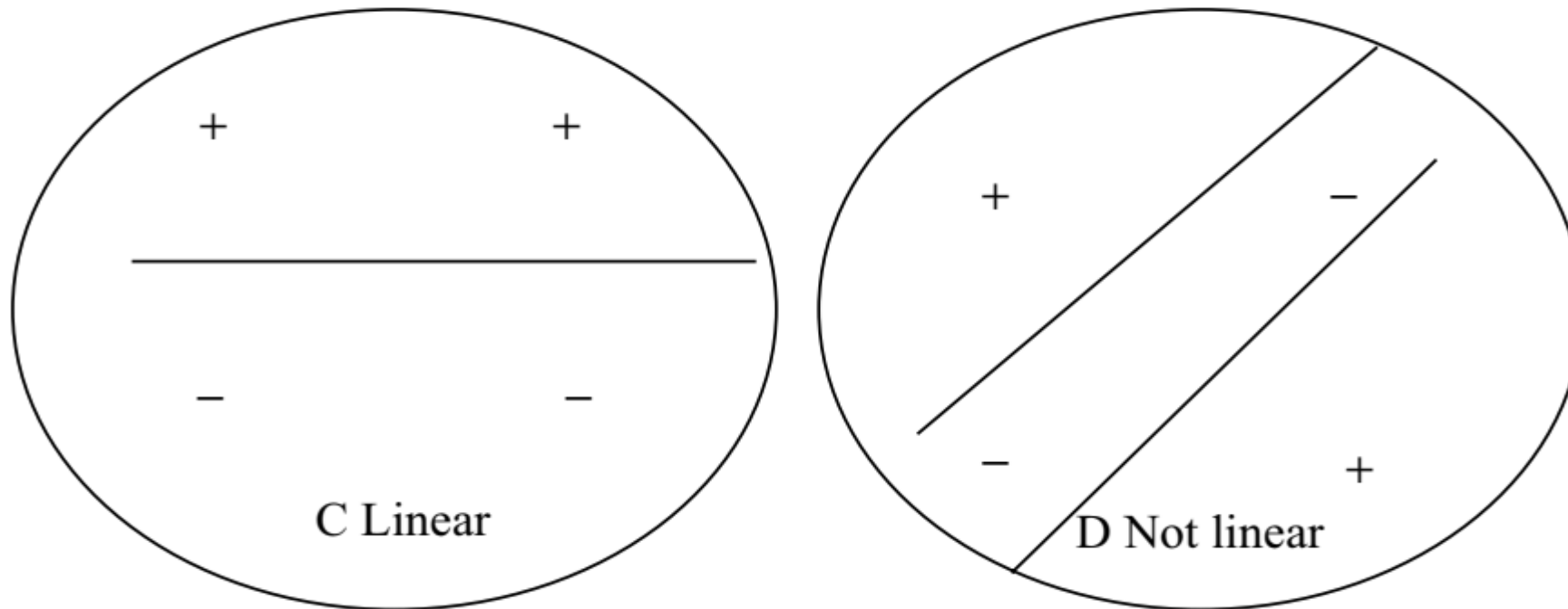
Perceptron Linear Separability

In a two-dimensional space, if two sets of points can be separated by a single line, then the sets satisfy the property of linear separability



Perceptron Linear Separability

In a two-dimensional space, if two sets of points can be separated by a single line, then the sets satisfy the property of linear separability



Preceptron Linear Separability

The perceptron we have discussed in the previous section for AND/OR logic problem, have the ability to satisfy the property of linear separability.

However, it cannot solve for the classification of XOR problem.

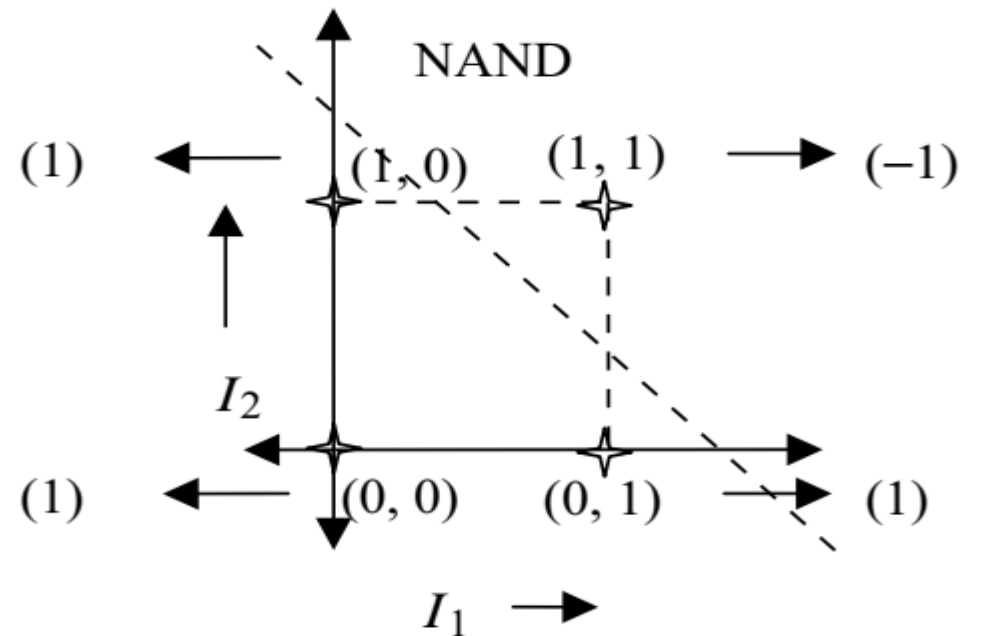
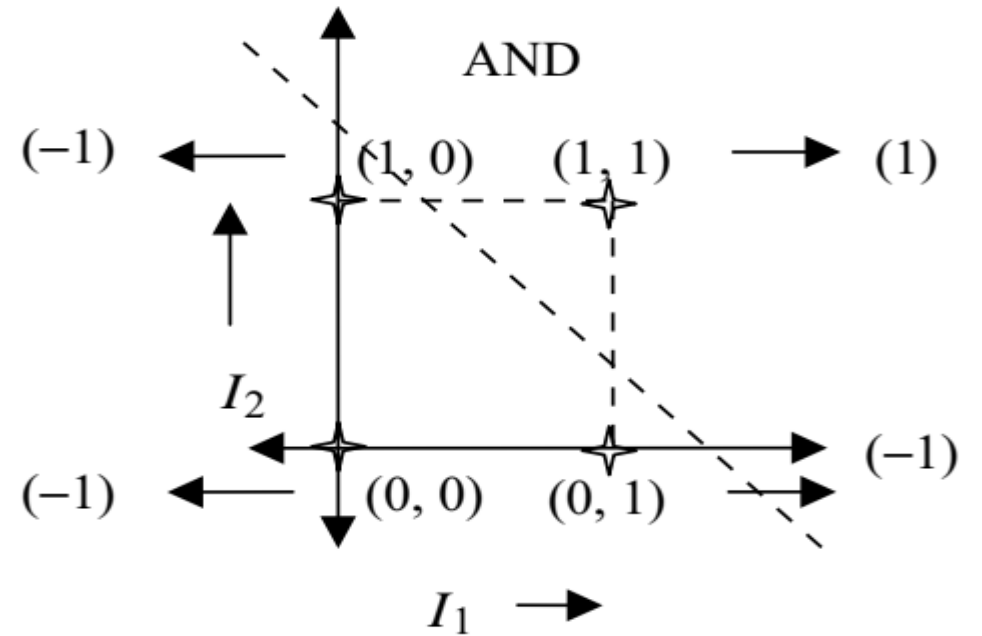
Table 2.9 Truth table of AND, OR, NAND, NOR, and XOR logic

Binary inputs (I)		Bipolar outputs				
		AND	OR	NAND	NOR	XOR
I_1	I_2	O_1	O_1	O_1	O_1	O_1
1	1	1	1	-1	-1	-1
1	0	-1	1	1	-1	1
0	1	-1	1	1	-1	1
0	0	-1	-1	1	1	-1

Preceptron Linear Separability

Table 2.9 Truth table of AND, OR, NAND, NOR, and XOR logic

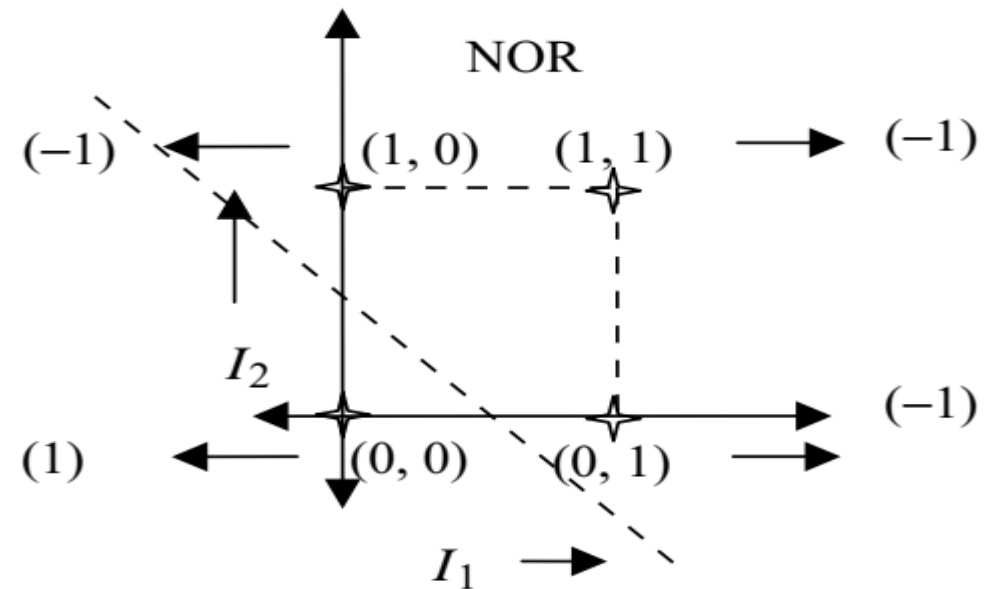
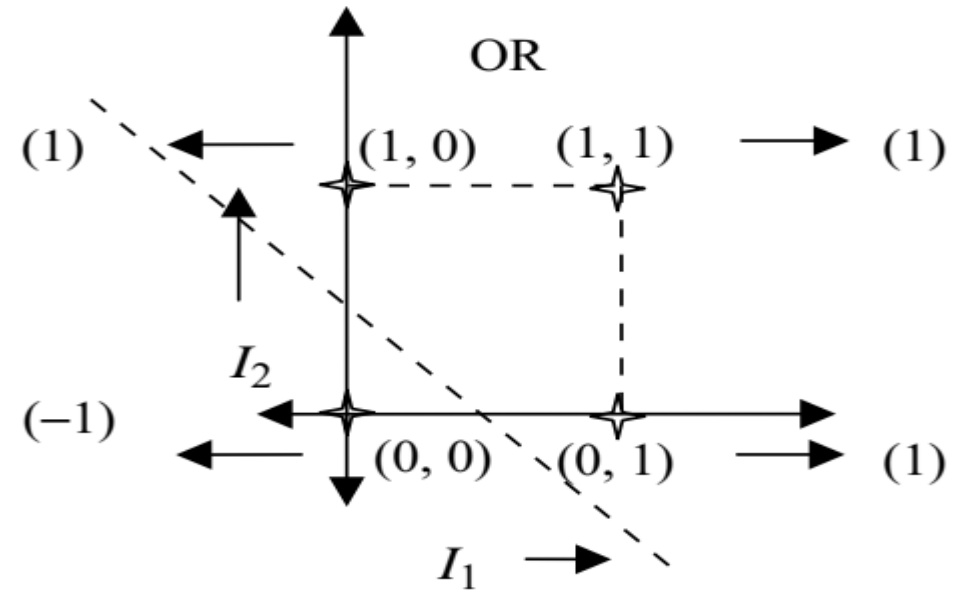
Binary inputs (I)		Bipolar outputs				
		AND	OR	NAND	NOR	XOR
I_1	I_2	O_1	O_1	O_1	O_1	O_1
1	1	1	1	-1	-1	-1
1	0	-1	1	1	-1	1
0	1	-1	1	1	-1	1
0	0	-1	-1	1	1	-1



Preceptron Linear Separability

Table 2.9 Truth table of AND, OR, NAND, NOR, and XOR logic

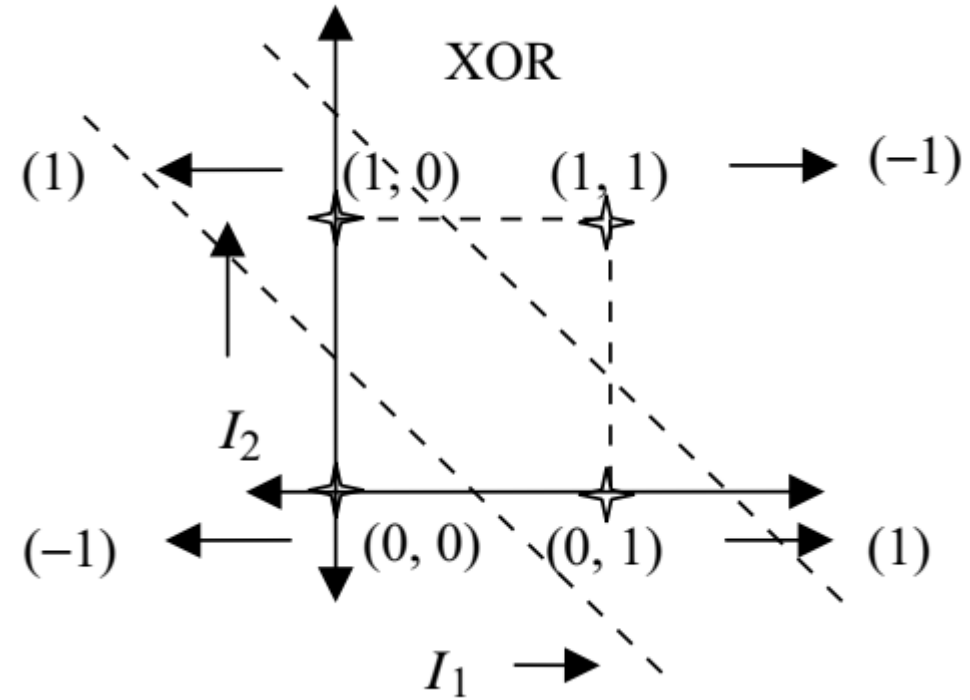
Binary inputs (I)		Bipolar outputs				
		AND	OR	NAND	NOR	XOR
I_1	I_2	O_1	O_1	O_1	O_1	O_1
1	1	1	1	-1	-1	-1
1	0	-1	1	1	-1	1
0	1	-1	1	1	-1	1
0	0	-1	-1	1	1	-1



Preceptron Linear Separability

Table 2.9 Truth table of AND, OR, NAND, NOR, and XOR logic

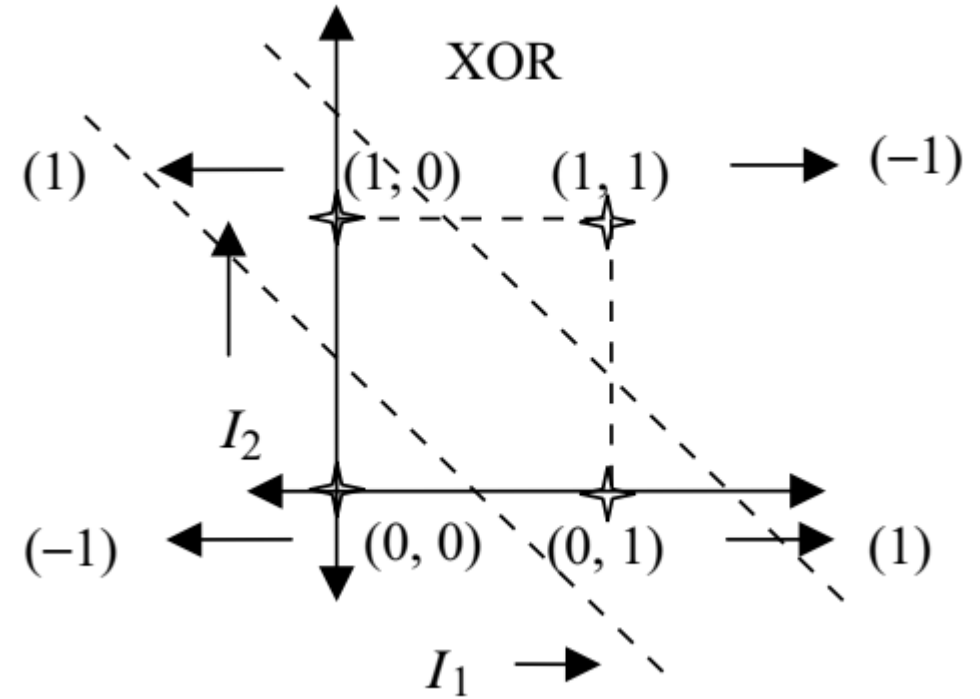
Binary inputs (I)		Bipolar outputs				
		AND	OR	NAND	NOR	XOR
I_1	I_2	O_1	O_1	O_1	O_1	O_1
1	1	1	1	-1	-1	-1
1	0	-1	1	1	-1	1
0	1	-1	1	1	-1	1
0	0	-1	-1	1	1	-1



Preceptron Linear Separability

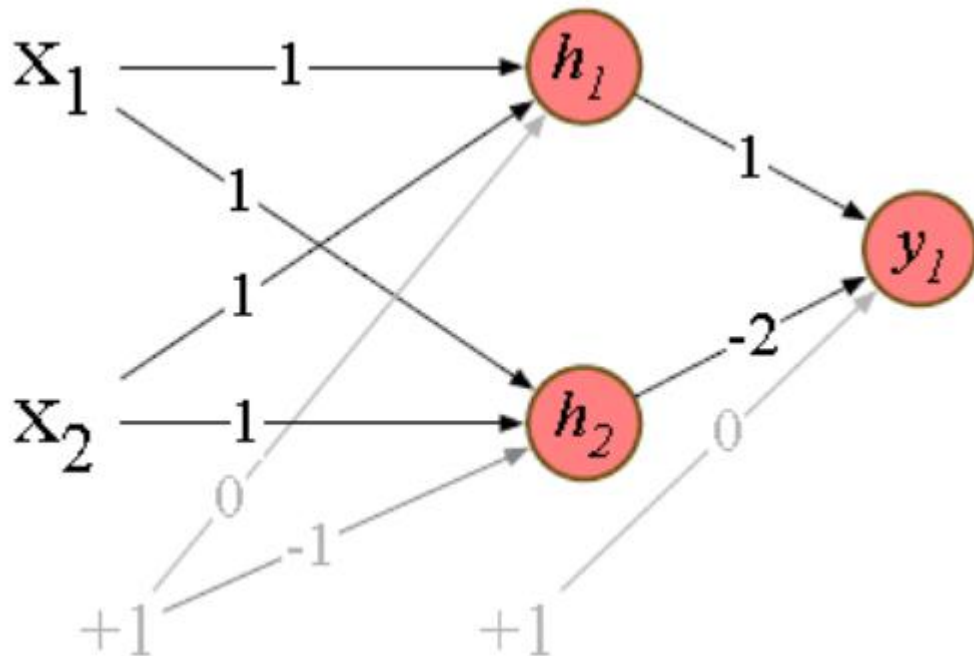
it is obvious that XOR problem cannot be classified into any of the two classes.

It is possible for the perceptron to get trained and behave such as AND, OR, NAND, and NOR logics, but it is impossible for the XOR problem.



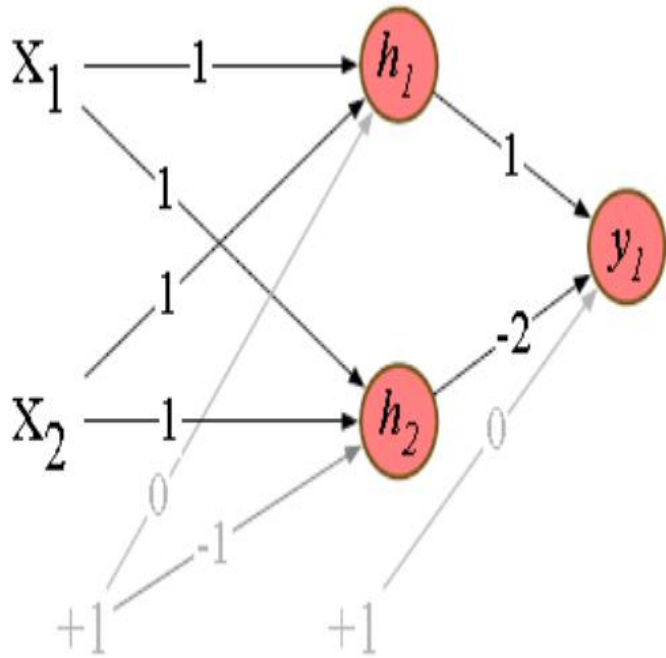
XOR Solution : Neural Network

- XOR function cannot be calculated by a single perceptron.
- **XOR using two layers of ReLU-based units**



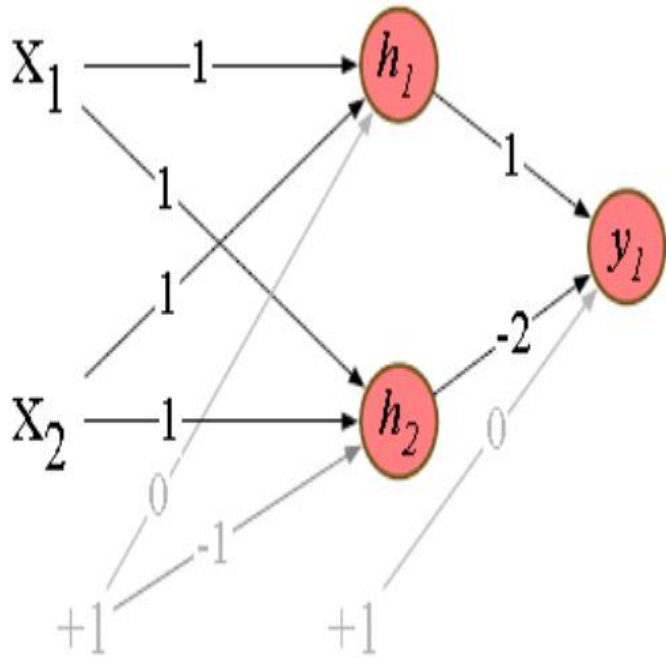
There are three ReLU units, in two layers; we've called them h_1 , h_2 (h for "hidden layer") and y_1 . bias weights/units in gray

XOR Solution : Neural Network



X1	X2	h1 = $W1x1+w2x2+b$ $W1=1, w2=1,$ $b=0$ Relu	h2= $W3x1+w4x2+b$ $W3=1, w4=1,$ $b=-1$ Relu	y1 = $w5*h1 + w6*h2$ $W5=1, w6 = -2$ $b=0$ Relu
0	0	$1*0+1*0+0 = 0$	$1*0+1*0-1 = -1$ Since Relu so output is 0	$1*0-2*0+0 = 0$
0	1	$1*0+1*1+0 = \mathbf{1}$	$1*0+1*1-1 = \mathbf{0}$	$1*\mathbf{1}-2*\mathbf{0}+0 = 1$
1	0	$1*1+1*0+0=1$	$1*1+1*0-1 = 0$	$1*1-2*0+0 = 1$
1	1	$1*1+1*1+0=2$	$1*1+1*1-1 = 1$	$1*1-2*1+0 = -1$ =0 since Relu

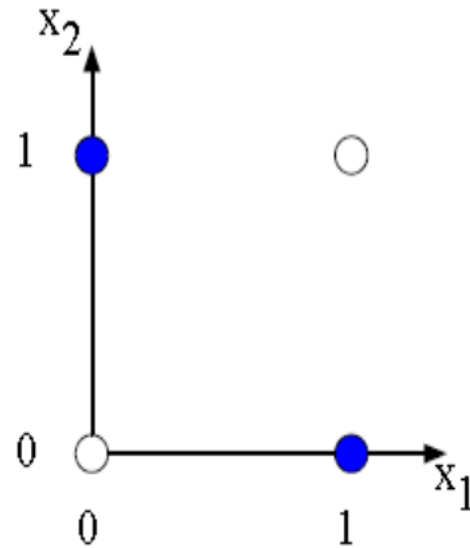
XOR Solution : Neural Network



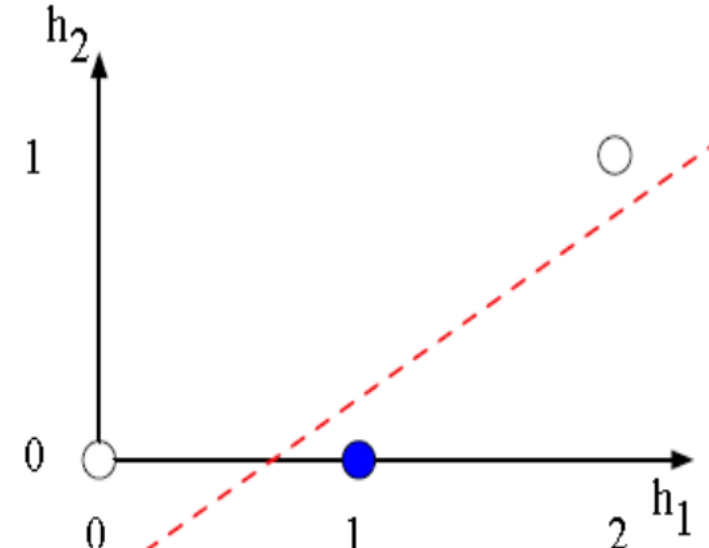
X_1	X_2	$h_1 =$ $W_1x_1 + W_2x_2 + b$ $W_1=1, W_2=1,$ $b=0$ Relu	$h_2 =$ $W_3x_1 + W_4x_2 + b$ $W_3=1, W_4=1,$ $b=-1$ Relu	$y_1 = w_5 \cdot h_1 + w_6 \cdot h_2$ $W_5=1, W_6 = -2$ $b=0$ Relu
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	2	1	0

XOR Solution : Neural Network

x_1	x_2	$h_1 =$ $W_1x_1 + w_2x_2 + b$ $W_1=1, w_2=1,$ $b=0$ <u>Relu</u>	$h_2 =$ $W_3x_1 + w_4x_2 + b$ $W_3=1, w_4=1,$ $b=-1$ <u>Relu</u>
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2	1



a) The original x space



b) The new (linearly separable) h space

The hidden layer forming a new representation of the input. (b) shows the representation of the hidden layer, h , compared to the original input representation x in (a). Notice that the input point $[0, 1]$ has been collapsed with the input point $[1, 0]$, making it possible to linearly separate the positive and negative cases of XOR

Preceptron

- **Training**

- for T iterations:

- * for each input vector:

- compute the activation of each neuron j using activation function g :

$$y_j = g \left(\sum_{i=0}^m w_{ij} x_i \right) = \begin{cases} 1 & \text{if } w_{ij} x_i > 0 \\ 0 & \text{if } w_{ij} x_i \leq 0 \end{cases}$$

- update each of the weights individually using:

$$w_{ij} \leftarrow w_{ij} + \eta(t_j - y_j) \cdot x_i$$

Preceptron (Logical OR)

in_1	in_2	t
0	0	0
0	1	1
1	0	1
1	1	1

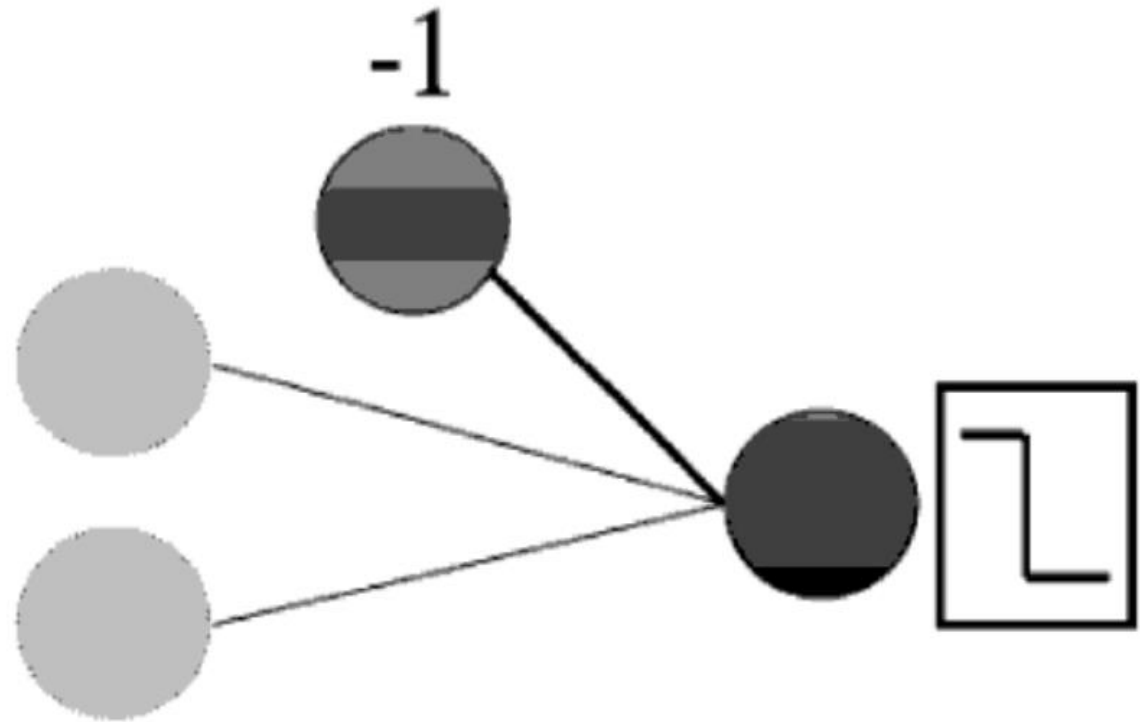


- compute the activation of each neuron j using:

$$y_j = g \left(\sum_{i=0}^m w_{ij} x_i \right) = \begin{cases} 1 & \text{if } w_{ij} x_i > 0 \\ 0 & \text{if } w_{ij} x_i \leq 0 \end{cases}$$

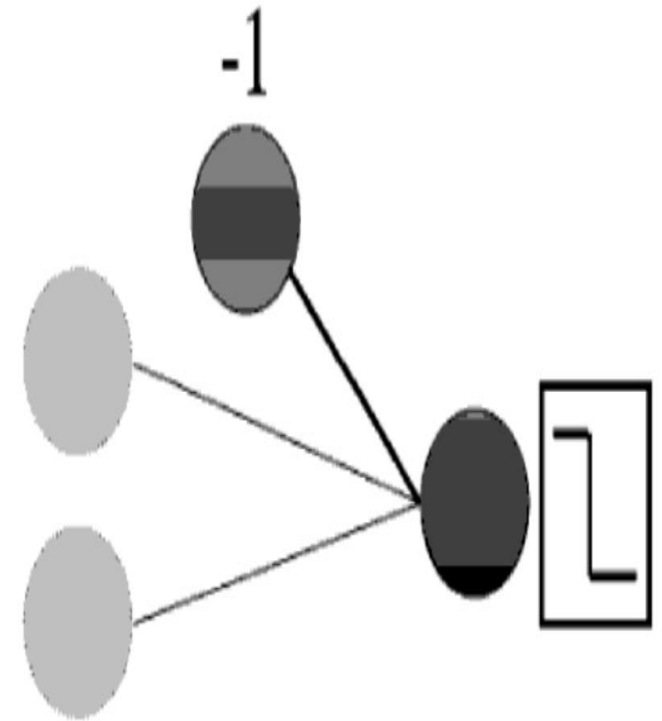
Example of Perceptron Learning

- **Logical OR**
- $(0, 0) \rightarrow 0$
- $(0, 1) \rightarrow 1$
- $(1, 0) \rightarrow 1$
- $(1, 1) \rightarrow 1$
- $W_0 = -0.05, w_1 = -0.02,$
- $w_2 = 0.02, \text{bias} = -1$
- Input $(0, 0)$



Example of Perceptron Learning

- **Logical OR**
 - **Input(0,0)→0**
 - $W_0 = -0.05$, $w_1 = -0.02$,
 - $w_2 = 0.02$, bias = -1
 - Input (0, 0)
 - Value that reaches neuron
 $= g(-0.05 \times -1 + -0.02 \times 0 + -0.02 \times 0) = g(0.05)$
 $= g(0.05) > 0$ so **neuron fires** and **output is 1**
- Which is incorrect so we need to update weight**



Example of Perceptron Learning

- **Logical OR**
- **Input(0,0)→0**
- $w_0 = -0.05, w_1 = -0.02,$
- $w_2 = 0.02, \text{ bias} = -1$
- Input (0, 0)
- $\eta = 0.25$

$$w_0 : -0.05 + 0.25 \times (0 - 1) \times -1 = 0.2,$$

$$w_1 : -0.02 + 0.25 \times (0 - 1) \times 0 = -0.02,$$

$$w_2 : 0.02 + 0.25 \times (0 - 1) \times 0 = 0.02.$$

Example of Preceptron Learning

- **Logical OR**
 - **Now Input(0,1)→1**
 - $w_0 = 0.2, w_1 = -0.02, w_2 = 0.02,$
 - bias = -1, $\eta = 0.25$
 - Input (0, 1)
 - Value that reaches neuron
 $= g(0.2 \times -1 + -0.02 \times 0 + 0.02 \times 1)$
 $= g(-0.18)$
 $= g(-0.18 < 0)$ so neuron not fires and **output is 0**
- Which is incorrect so we no need to update weight**

$$w_0 : 0.2 + 0.25 \times (1 - 0) \times -1 = -0.05,$$
$$w_1 : -0.02 + 0.25 \times (1 - 0) \times 0 = -0.02,$$
$$w_2 : 0.02 + 0.25 \times (1 - 0) \times 1 = 0.27.$$

Example of Perceptron Learning

- **Logical OR**
 - **Now Input(1,0)→1**
 - $W_0 = -0.05, w_1 = -0.02, w_2 = 0.27,$
 - $\text{bias} = -1$
 - Value that reaches neuron
 $= g(-0.05 \times -1 + -0.02 \times 1 + 0.27 \times 0)$
 $= g(0.05 - 0.02 = 0.03 > 0)$
so neuron fires and **output is 1**
- Which is correct so we no need to update weight**

Example of Perceptron Learning

- **Logical OR**
- **Now Input(1,1)→1**
- $W_0 = -0.05, w_1 = -0.02, w_2 = 0.27, \text{ bias} = -1$
- Value that reaches neuron

$$= g(-0.05 \times -1 + -0.02 \times 1 + 0.27 \times 1)$$

$$= g(0.05 - 0.02 + 0.27 = 0.30 > 0)$$

so neuron fires and **output is 1, Which is correct so no need to update weight**

Example of Perceptron Learning

We have not finished all the cases (not all the answers are correct)

We now need to start going through the input again, until the weight settle down and stop changing.

Example with two outputs

- Input $x_1=2, x_2=1, x_3=3, t_1=1; t_2=0$
- Let $W_{01}=0.5, w_{02}=-0.5,$
- $W_{11}=0.1, w_{12}=0$
- $w_{21}=0.4, w_{22}=-0.2$
- $w_{31}=-0.1, w_{32}=-0.1$

- In matrix form, $W = \begin{bmatrix} 0.5 & -0.5 \\ 0.1 & 0 \\ 0.4 & -0.2 \\ -0.1 & -0.1 \end{bmatrix}, X = \begin{bmatrix} 1 \\ 2 \\ 1 \\ 3 \end{bmatrix}, Y = g(w^t X)$

Example

- In matrix form, $W = \begin{bmatrix} 0.5 & -0.5 \\ 0.1 & 0 \\ 0.4 & -0.2 \\ -0.1 & -0.1 \end{bmatrix}$, $X = \begin{bmatrix} 1 \\ 2 \\ 1 \\ 3 \end{bmatrix}$, $Y = g(w^t X)$

$$g \begin{bmatrix} 0.8 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Correct output since input is (1, 0) and desired output (1,0) is also same so no need to change weight

Another Example

- Input $x_1=2, x_2=1, x_3=3, \mathbf{t_1=1; t_2=0}$

- In matrix form, $W = \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 0 \\ 0.4 & -0.2 \\ -0.1 & -0.1 \end{bmatrix}, X = \begin{bmatrix} 1 \\ 2 \\ 1 \\ 3 \end{bmatrix},$

- $Y = g(w^t X)$

$$g \begin{bmatrix} -0.4 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Not Correct output since input is (1, 0) and output is (0, 0) so we need to change weight

$$y_j = g \left(\sum_{i=0}^m w_{ij} x_i \right) = \begin{cases} 1 & \text{if } w_{ij} x_i > 0 \\ 0 & \text{if } w_{ij} x_i \leq 0 \end{cases}$$

- update each of the weights individually using:

$$w_{ij} \leftarrow w_{ij} + \eta(t_j - y_j) \cdot x_i$$

Change Weight

$N=0.3,$

$$w^t = \begin{bmatrix} 0.5 & -0.5 & 0.4 & -0.1 \\ -0.5 & 0 & -0.2 & -0.1 \end{bmatrix}$$

$$X^t = [1 \ 2 \ 1 \ 3]$$

$$w^t = w^{t-1} + \eta(t-y) X^t$$

$$w^t = \begin{bmatrix} 0.5 & -0.5 & 0.4 & -0.1 \\ -0.5 & 0 & -0.2 & -0.1 \end{bmatrix} + 0.3$$

$$\begin{bmatrix} 1 & -0 \\ 0 & -0 \end{bmatrix} [1 \ 2 \ 1 \ 3]$$

$$Y = g(w^t X)$$

$$g \begin{bmatrix} 4.1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \text{Now Correct Output}$$

Classification by backpropagation

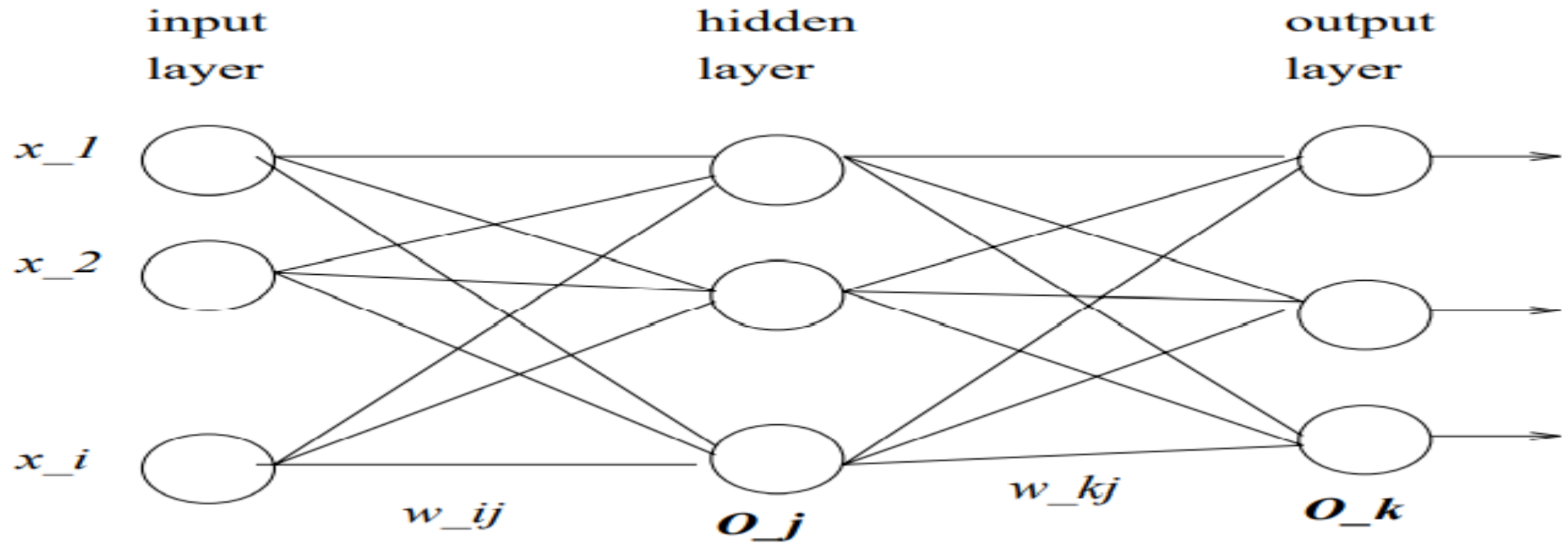
- Backpropagation is a neural network learning algorithm.
- neural network is a set of connected input/output units where each connection has a weight associated with it.
- During the learning phase, the network learns by adjusting the weights so as to be able to predict the correct class label of the input samples.
- Neural networks involve long training times
- They require a number of parameters which are typically best determined empirically

- Neural networks have been criticized for their poor interpretability.
- **Advantages of neural networks,**
- however, include their high tolerance to noisy data
- their ability to classify patterns on which they have not been trained.
- several algorithms have recently been developed for the extraction of rules from trained neural networks.
- These factors contribute towards the usefulness of neural networks for classification in data mining.

A multilayer feed-forward neural network

- The backpropagation algorithm performs learning on a multilayer feed-forward neural network.
- **The inputs correspond to the attributes** measured for each training sample.
- The **inputs are fed simultaneously** into a layer of units making up the **input layer**.
- **The weighted outputs** of these units are, in turn, fed simultaneously to a second layer of “neuron-like” units, known as a **hidden layer**

Classification by backpropagation



A multilayer feed-forward neural network: A training sample, $\mathbf{X} = (\mathbf{x}_1; \mathbf{x}_2; \dots; \mathbf{x}_i)$, is fed to the input layer. Weighted connections exist between each layer, where w_{ij} denotes the weight from a unit j in one layer to a unit i in the previous layer.

A multilayer feed-forward neural network

- hidden layer's weighted outputs can be input to another hidden layer, and so on.
- number of hidden layers is arbitrary, usually only one is used.
- weighted outputs of last hidden layer are input to units making up output layer, which emits the network's prediction for given samples

A multilayer feed-forward neural network

- The multilayer neural network shown in Figure has two layers of output units.
- Therefore, we say that it is a two-layer neural network.
- Similarly, a network containing two hidden layers is called a three-layer neural network, and so on.
- It is fully connected in that each unit provides input to each unit in the next forward layer.
-

Defining a network topology

- user must decide on the network topology by specifying the number of units in the input layer, the number of hidden layers (if more than one), the number of units in each hidden layer, and the number of units in the output layer.
- **Normalizing the input values** for each attribute measured in the training samples will help **speed up the learning phase**.
- Typically, input values are normalized so as to fall **between 0 and 1.0**.

Defining a network topology

- Discrete-valued attributes may be encoded such that there is one input unit per domain value.
- For example, if the domain of an attribute A is $\{a_0, a_1, a_2\}$,
- then we may assign three input units to represent A .
- That is, we may have, say, $I_0; I_1; I_2$, as input units
- Each unit is initialized to 0.
- If $A = a_0$, then I_0 is set to 1.
- If $A = a_1$, I_1 is set to 1, and so on

Backpropagation

- Backpropagation learns by iteratively processing a set of training samples,
- comparing the network's prediction for each sample with the actual known class label.
- For each training sample, the weights are modified so as to minimize the mean squared error between the network's prediction and the actual class.
- These modifications are made in the “backwards” direction, i.e., from the output layer, through each hidden layer down to the first hidden layer

Backpropagation

- Initialize the weights.
- The weights in the network are initialized to small random numbers (e.g., ranging from -1.0 to 1.0, or -0.5 to 0.5).
- Each unit has a bias associated with it, as explained below.
- The biases are similarly initialized to small random numbers

Backpropagation

- **Propagate the inputs forward.**
- In this step, the net input and output of each unit in the hidden and output layers are computed.
- First, the training sample is fed to the input layer of the network.
- The net input to each unit in the hidden and output layers is then computed as a linear combination of its inputs.

Backpropagation

- **Propagate the inputs forward.**
- *Given a unit j in a hidden or output layer, the net input, I_j , to unit j is:*
- $I_j = \sum_i w_{ij} O_i + \theta_j$
- Where O_i is the output of Unit i from previous layer
- θ_j is bias of the unit
- w_{ij} is the weight of the connection from unit i in the previous layer to unit j ;

Backpropagation

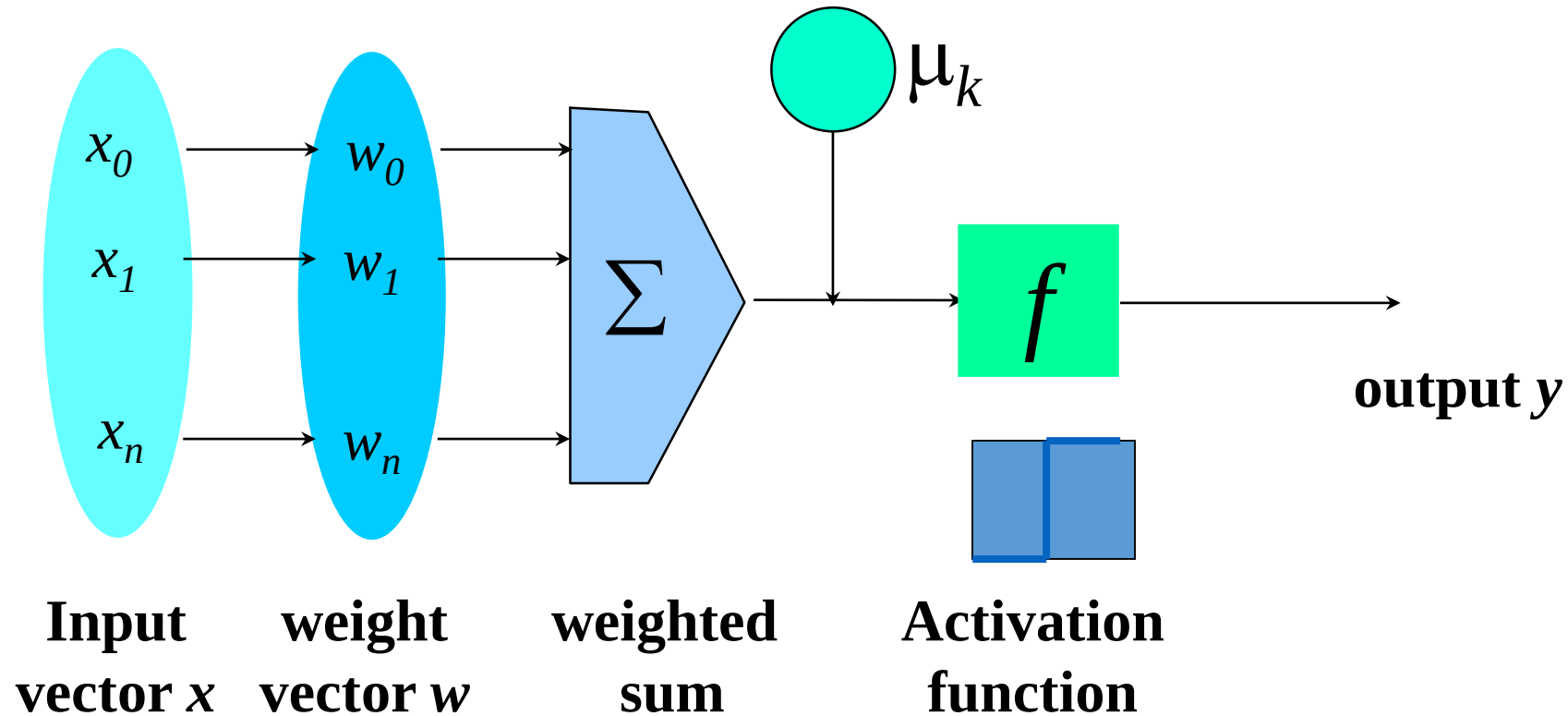
- Each unit in the hidden and output layers takes its net input, and then applies an activation function to it.
- The function symbolizes the activation of the neuron represented by the unit. The logistic, or sigmoid function is used.
- Given the net input I_j to unit j , then O_j , the output of unit j , is computed as:
- $$O_j = \frac{1}{1 + e^{-I_j}}$$
- *This function is also referred to as a squashing function, since it maps a large input domain onto the smaller range of 0 to 1*

- **Backpropagate the error**
- The error is propagated backwards by updating the weights and biases to reflect the error of the network's prediction.
- *Out Put Layer* = $Err_j = O_j (1 - O_j) (T_j - O_j)$
- *Hidden Layer* = $Err_j = O_j (1 - O_j) \sum_k Err_k w_{jk}$
- where O_j is the actual output of unit j , and T_j is the true output, based on the known class label of the given training sample.

- Weights are updated by
- $w_{ij} = w_{ij} + \nabla w_{ij}$
- $\nabla w_{ij} = (l)Err_j O_i$
- Biases are updated by
- $\theta_j = \theta_j + \nabla \theta_j$
-
- $\nabla \theta_j = (l) Err_j$

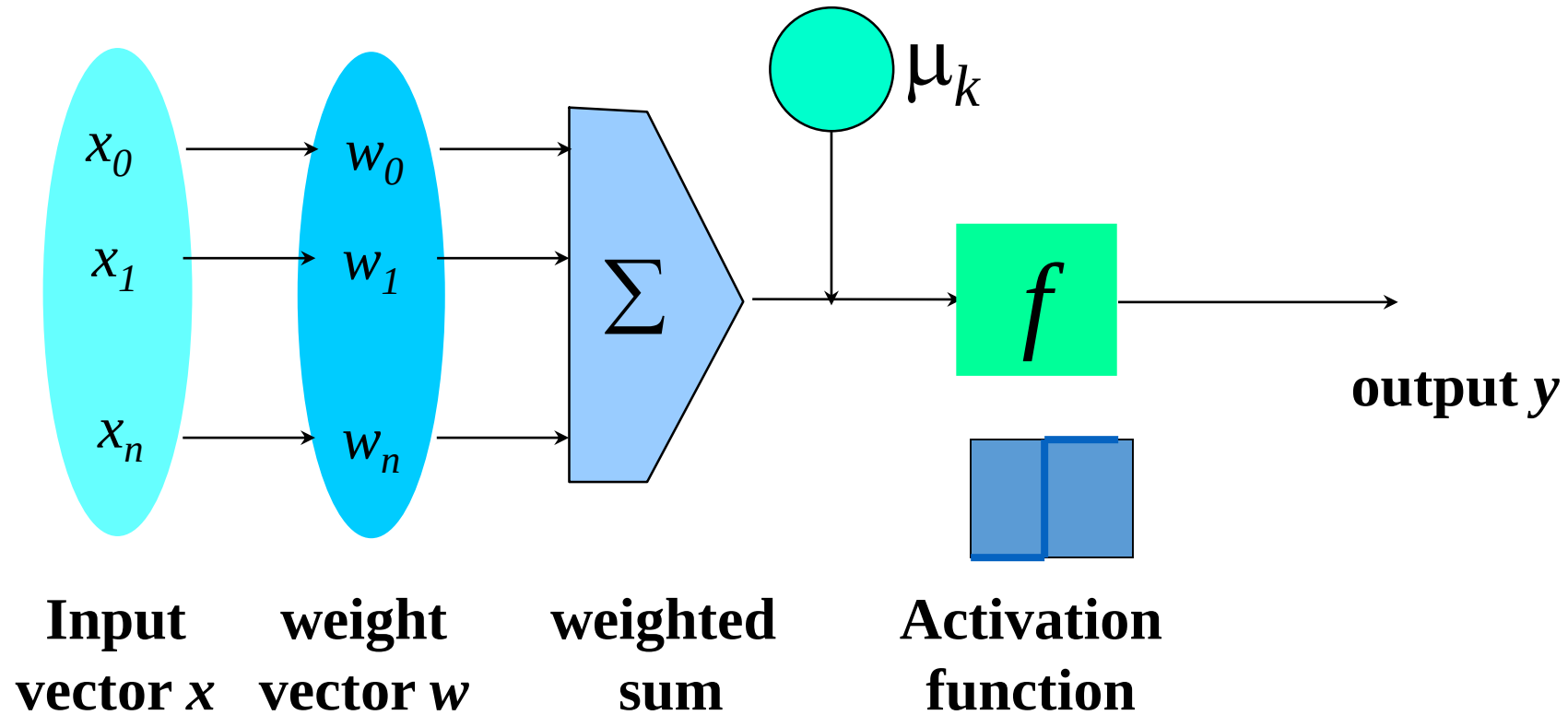
- ***Training stops when either***
- all w_{ij} in the previous epoch were so small as to be below some specified threshold, or
- the percentage of samples misclassified in the previous epoch is below some threshold, or
- a prespecified number of epochs has expired

A Neuron



- The n -dimensional input vector x is mapped into variable y by means of the scalar product and a nonlinear function mapping

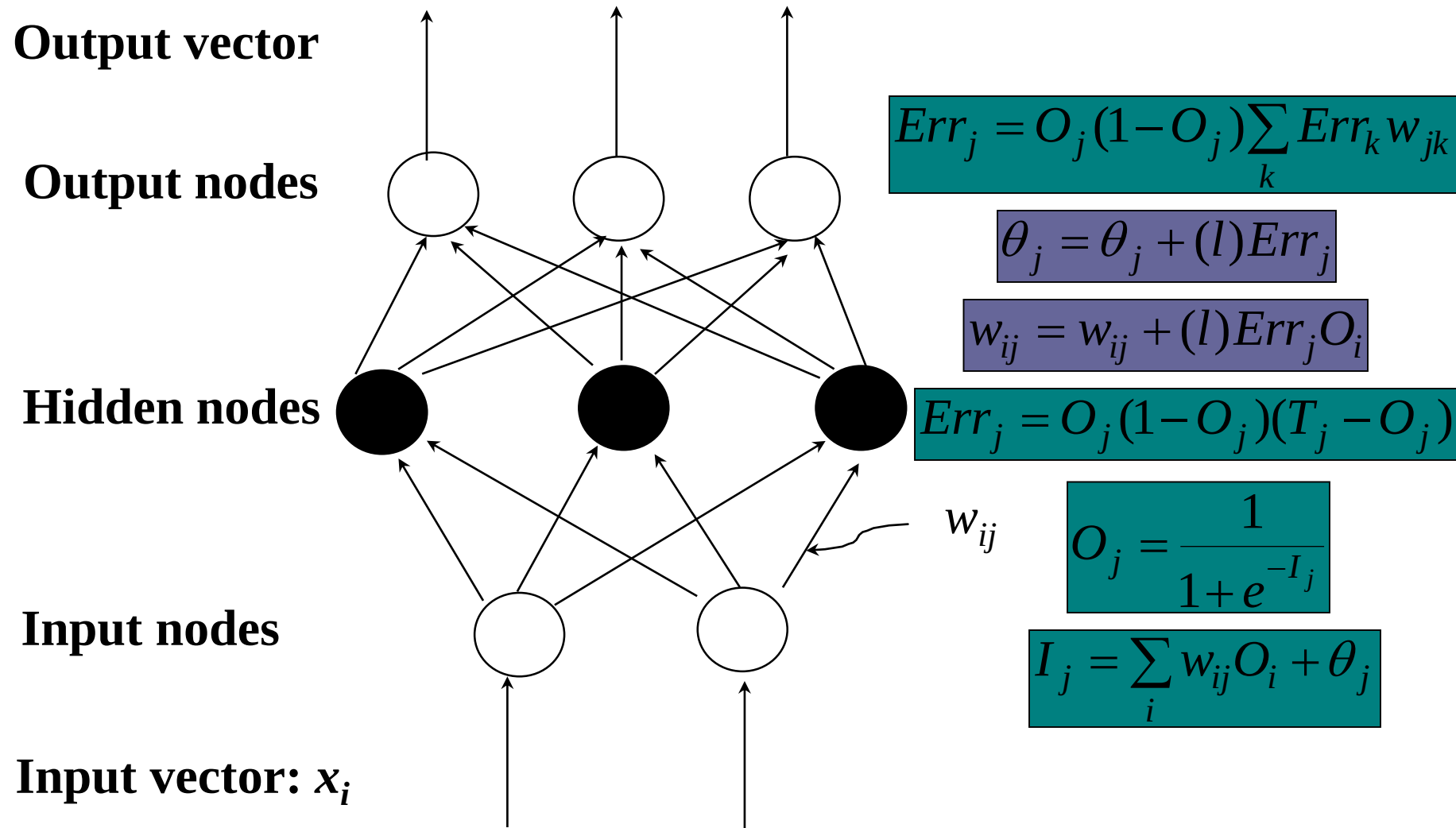
A Neuron



For Example

$$y = \text{sign}\left(\sum_{i=0}^n w_i x_i + \mu_k\right)$$

Multi-Layer Perceptron



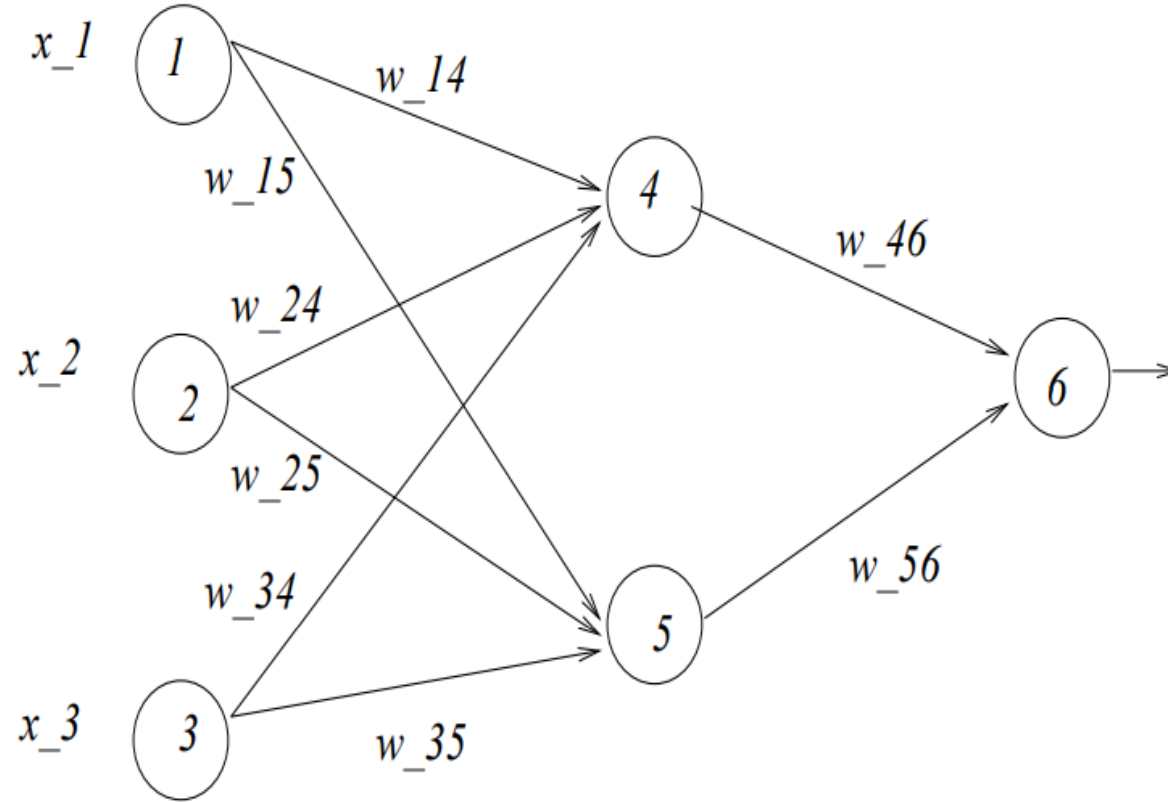


Figure 7.12: An example of a multilayer feed-forward neural network.

- *Example 7.5 Sample calculations for learning by the backpropagation algorithm. Figure 7.12 shows a multilayer feed-forward neural network. The initial weight and bias values of the network are given in Table 7.3, along with the first training sample, $X = (1; 0; 1)$.*

x1	x2	x3	w14	w15	w24	w25	w34	w35	w46	w56	θ_4	θ_5	θ_6
1	0	1	0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

x1	x2	x3	w14	w15	w24	w25	w34	w35	w46	w56	θ_4	θ_5	θ_6
1	0	1	0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

Unit j	Net Input, $I_j = I_j = \sum_i w_{ij} O_i + \theta_j$ Where O_i is the output of Unit i from previous layer, X(1, 0, 1) $O_1 = 1, O_2 = 0, O_3 = 1,$	Output, $O_j = \frac{1}{1 + e^{-I_j}}$
4	For node 4, $w_{14}=0.2, w_{24}=0.4$ and $w_{34}=-0.5$ $= w_{14} * O_1 + w_{24} * O_2 + w_{34} * O_3 + \theta_4$ $= 0.2 * 1 + 0.4 * 0 + (-0.5) * 1 - 0.4$ $= -0.7$	$O_4 = \frac{1}{1 + e^{0.7}} = 0.33$
5	For node 5, $w_{15}=-0.3, w_{25}=0.1$ and $w_{35}=0.2$ $= w_{15} * O_1 + w_{25} * O_2 + w_{35} * O_3 + \theta_5$ $= -0.3 * 1 + 0.1 * 0 + 0.2 * 1 + 0.2$ $= 0.1$	Output, $O_5 = \frac{1}{1 + e^{-0.1}} = 0.52$
6	For node 6, $w_{46}=-0.3, w_{56} = -0.2, \theta_6 = 0.1, O_4 = 0.33, O_5 = 0.52$ $= w_{46} * O_4 + w_{56} * O_5 + \theta_6$ $= -0.3 * 0.33 + (-0.2) * 0.52 + 0.1$ $= 0.19$	Output, $O_6 = \frac{1}{1 + e^{-0.19}} = 0.55$

Unit j	$\text{Out Put Layer} = \text{Err}_j = O_j (1 - O_j) (T_j - O_j)$ $\text{Hidden Layer} = \text{Err}_j = O_j (1 - O_j) \sum_k \text{Err}_k w_{jk}$
6	$O_6 = 0.55, T_6 = 1$ $\text{Err}_6 = (0.55)(1-0.55)(1-0.55) = 0.495$ $\text{Out Put Layer} = \text{Err}_j = O_j (1 - O_j) (T_j - O_j)$
5	$\text{Hidden Layer} = \text{Err}_j = O_j (1 - O_j) \sum_k \text{Err}_k w_{jk}$ K=5, $O_5 = 0.52$, $\text{Err}_6 = 0.495$ from previous unit, $w_{56} = -0.3$ $(0.52) (1-0.52) (0.495) (-0.3) = 0.037$
4	$\text{Hidden Layer} = \text{Err}_j = O_j (1 - O_j) \sum_k \text{Err}_k w_{jk}$ K=4, $O_4 = 0.33$, $\text{Err}_6 = 0.495$ from previous unit, $w_{46} = -0.2$ $(0.33) (1-0.33) (0.495) (-0.2) = -0.022$



Weight or Bias	New Value $w_{ij} = w_{ij} + \nabla w_{ij}$ $\nabla w_{ij} = (l)Err_j O_i$
w_{46}	$i = 4, j = 6, w_{46} = -0.3, l = 0.9, Err_6 = 0.495, O_i = O_4 = 0.33$ $= -0.3 + (0.9)(0.495)(0.33) = -0.153$
w_{56}	$i = 5, j = 6, w_{56} = -0.3, l = 0.9, Err_6 = 0.495, O_i = O_5 = 0.52$ $= -0.2 + (0.9)(0.495)(0.52) = -0.032$
w_{14}	$i = 1, j = 4, w_{14} = 0.2, l = 0.9, Err_4 = -0.022, O_i = O_1 = 1$ $= 0.2 + (0.9)(-0.022)(1) = 0.180$
w_{15}	$i = 1, j = 5, w_{15} = -0.3, l = 0.9, Err_5 = 0.037, O_i = O_1 = 1$ $= -0.3 + (0.9)(0.037)(1) = -0.267$
w_{24}	$i = 2, j = 4, w_{24} = 0.4, l = 0.9, Err_4 = -0.022, O_i = O_2 = 0$ $= 0.4 + (0.9)(-0.022)(0) = 0.4$
w_{25}	$i = 2, j = 5, w_{25} = 0.1, l = 0.9, Err_5 = 0.037, O_i = O_2 = 0$ $= 0.1 + (0.9)(0.037)(0) = 0.1$

w_{34}	$i = 3, j = 4, w_{34} = -0.5, l = 0.9, Err_4 = -0.022, O_i = O_3 = 1$ $= -0.5 + (0.9)(-0.022)(1) = -0.520$
w_{35}	$i = 3, j = 5, w_{35} = 0.1, l = 0.9, Err_5 = 0.037, O_i = O_3 = 1$ $= 0.1 + (0.9)(0.037)(1) = 0.233$
	$\theta_j = \theta_j + \nabla \theta_j$ $\nabla \theta_j = (l) Err_j$
θ_6	$\theta_6 = 0.1, Err_6 = 0.495$ $\theta_6 = 0.1 + (0.9)(0.495) = 0.546$
θ_5	$\theta_5 = 0.2, Err_5 = 0.037$ $\theta_5 = 0.2 + (0.9)(0.037) = 0.233$
θ_4	$\theta_4 = 0.4, Err_4 = -0.022$ $\theta_4 = 0.4 + (0.9)(-0.022) = -0.420$

Application of ANN

- **Speech Recognition**

Following ANNs have been used for speech recognition –

- Multilayer networks
- Multilayer networks with recurrent connections
- Kohonen self-organizing feature map

- **Character Recognition**

- Following are some ANNs which have been used for character recognition
- Multilayer neural networks such as Backpropagation neural networks.
- Neocognitron

Application of ANN

- **Signature Verification Application**

Convolution Neural Network

- **Human Face Recognition**
- Fully-connected multilayer feed-forward neural network trained with the help of back-propagation algorithm.
- For dimensionality reduction, Principal Component Analysis PCA is used.
- **Convolution Neural Network**
- **Stock Market Prediction**

Using Multilayer Perceptron MLP (*class of feedforward artificial intelligence algorithm*)

Application of ANN

- **Social Media**
 - to study the behaviours of social media users.
 - **Multilayer Perceptron ANN** is used to mine data from social media applications.
- **Aerospace**
 - developments in spacecraft and aircraft. Fault diagnosis, high performance auto piloting, securing the aircraft control systems, and modeling key dynamic simulations are some of the key areas that neural networks have taken over. Time delay Neural networks can be employed for modelling non linear time dynamic systems.

Application of ANN

- **Defence**
 - used in logistics, armed attack analysis, and for object location.
 - They are also used in air patrols, maritime patrol, and for controlling automated drones
- **Healthcare**
 - **Convolutional Neural Networks** are actively employed in the healthcare industry for **X ray detection, CT Scan** and **ultrasound**.
 - **Recurrent Neural Network (RNN)** is also being employed for the development of voice recognition systems.

Application of ANN

- **Weather forecasting**
- Multilayer Perceptron (MLP), Convolutional Neural Network (CNN) and Recurrent Neural Networks (RNN) are used for weather forecasting.

References

- Padhy, N.P.; Simon, S.P.. Soft Computing with MATLAB Programming.
- Engelbrecht Andries P., Computational Intelligence: An Introduction, Wiley
- Stephen Marsland, Chapman & Hall/CRC, Machine Learning An Algorithmic Prospective
- Online
- <https://web.stanford.edu/%7Ejurafsky/slp3/7.pdf>
- <https://www.analyticssteps.com/blogs/8-applications-neural-networks>
- <https://www.geeksforgeeks.org/activation-functions-neural-networks/>
- <https://www.geeksforgeeks.org/activation-functions-in-neural-networks-set2/>
- <https://www.geeksforgeeks.org/gradient-descent-algorithm-and-its-variants/>